Observability: LangFuse y LangSmith

Master AI Engineer · Sesión 6 · Ignacio de Pastors

thePower Tech School

AGENDA

01 Bloque teórico
02 Bloque práctico
03 Ejercicio para casa
04 Cierre y dudas



thePower
Tech School

01

Bloque teórico

thePower
Tech School

Por qué observabilidad: abrir la caja negra

Un agente no es una llamada: son 5, 10 o 20 pasos ocultos entre la pregunta y la respuesta.

- **Depurar a ciegas**: si la respuesta es mala, no sabes si falló el **prompt**, el **retrieval** o el **modelo**
- **Calidad = opinión**: "A mí me funcionó" no es una **métrica que puedas defender**
- **Coste invisible**: pagas **por token** y no sabes en qué se va el dinero
- **Latencia opaca**: el usuario dice "Va lento" y no sabes **qué paso lo causa**

CASO REAL: chatbot de soporte, la factura pasa de **200 a 800 EUR** en un mes. Sin trazas no sabes que el culpable era un prompt de sistema de **12.000 tokens** repetido en cada turno.

ANALOGÍA

La observabilidad es la **caja negra de un avión**.
Graba cada instrumento y cada decisión durante todo el vuelo. Mientras todo va bien, nadie la mira.
Cuando algo falla, la abres, rebobinas y ves el paso exacto en el que se torció: qué prompt entró, qué documento se recuperó, qué respondió el modelo.
En un demo te lo perdonas. En producción, volar sin caja negra es **inaceptable**.

thePower Tech School

Traces, spans y generations: el árbol de una ejecución

TRACE · la ejecución completa, de la pregunta del usuario a la respuesta final

SPAN · un paso dentro de ella: un retrieval, una tool o un nodo de tu grafo

GENERATION · una llamada al LLM: prompt enviado, respuesta, tokens, coste y latencia

Es un árbol y se anida: el trace contiene spans, y los spans contienen generations.
CASO REAL: "¿Cuántos días de vacaciones me quedan?" = **1 trace** > span de búsqueda + span de tool + span de redacción con 1 generation. Si el dato sale mal, abres el span de búsqueda y lo ves.

thePower Tech School

Qué registra cada llamada al LLM

Cada generation guarda mucho más que la pregunta y la respuesta:

- **Tokens de entrada y salida** - **15.000** de entrada = estás enviando historial de más
- **Coste en euros** de esa llamada concreta, con la tarifa del modelo ya aplicada
- **Latencia por paso** - responde si lo lento es el LLM, el retrieval o tu API
- **Modelo y parámetros** - temperatura, max tokens, versión exacta del modelo

Preguntas que antes no podías responder: ¿cuál es mi paso más lento? ¿Cuánto cuesta una conversación? ¿Qué usuario se come el presupuesto?

EJEMPLO

Una pregunta = 1 trace, 3 spans (retrieval, LLM, tool), **1.850 tokens** y **0,004 EUR**.

Cuatro milésimas parecen nada... hasta que multiplicas por **10.000 conversaciones** al mes: **40 EUR**, y ahora sabes exactamente de dónde salen.

El envío va en segundo plano: no ralentiza tu app, y puedes muestrear si tienes mucho tráfico.

thePower Tech School

Prompt management: sacar los prompts del código

El prompt deja de estar '**quemado**' entre comillas triples en tu código.

- **Se guarda y versiona** en la plataforma: cada cambio deja versión, fecha y autor
- **Lo editas sin desplegar**: cambio de 30 segundos, no despliegue de 20 minutos
- **Lo edita quien sabe**: negocio o soporte tocan el tono sin entrar al repositorio
- **Comparas versiones** y haces rollback en un clic si la nueva empeora

CASO REAL: marketing suaviza el tono del asistente para Navidad, se publica la v4 y el agente cambia esa misma tarde. Nadie ha desplegado nada.

CÓDIGO

```
prompt = client.get_prompt(
'chatbot', version=2)
```

Cambias la versión, no el código.

El SDK cachea el prompt en memoria: si la plataforma cae, tu app sigue con la última versión conocida. Aun así, deja siempre un prompt por defecto como red de seguridad.

thePower Tech School

Evaluación: medir en vez de opinar

01

Dataset
20-30 casos reales, nacidos de fallos reportados

02

Ejecutar
el agente sobre todos, de forma automática

03

Evaluar
reglas para lo objetivo, LLM-juez para lo subjetivo

04

Métricas
% acierto, coste y latencia por versión

Sin evaluación vas a ciegas: el modelo no es determinista y tu memoria de "cómo respondía antes" es mala.
CON DATOS: "La v3 acierta el **90%** frente al **80%** de la v2, cuesta un **20%** menos y responde **0,5 s** antes". Empieza con 10 ejemplos hoy, no con los 100 perfectos que nunca harás.

thePower Tech School

LLM-as-a-judge y evaluadores

POR REGLAS

- Comprobaciones **exactas y deterministas**, escritas en código
- ¿Contiene el enlace de cancelación? ¿Es un JSON válido? ¿Menos de 200 palabras?
- **Coste cero** y resultado inmediato: ideal para formato y seguridad
- **Límite**: una regla no sabe si la respuesta es buena, solo si cumple

Úsalas cuando la salida alimenta a otro sistema.

LLM COMO JUEZ

- Otro modelo puntúa **del 1 al 5** la respuesta del primero
- Le pasas pregunta, respuesta y (si la hay) la ideal, y le pides **nota + justificación**
- Mide lo que la regla no ve: **calidad, tono, relevancia, alucinaciones**
- **Se calibra**: puntúas 20 casos a mano y compruebas que el juez coincide

En la práctica, se combinan las dos.

thePower Tech School

LangFuse vs LangSmith

LangFuse

- **Open source** y **self-host gratis**: tus datos no salen de tu máquina
- Imprescindible en **banca o salud**, donde el dato no puede salir
- **Agnóstica**: LangChain, LlamaIndex, SDK de OpenAI o código tuyo
- Instrumentación explícita: decoras tus funciones con **@observe**

Gana en control, portabilidad y privacidad.

LangSmith

- **Nativa de LangChain / LangGraph**: entiende nodos, aristas y estado
- Se activa con **una variable de entorno**, sin tocar una línea de código
- Primera traza en **2 minutos**: el momento en que se enciende la bombilla
- Hosted en su nube, con **free tier** de sobra para este curso

Gana en comodidad e integración.

thePower Tech School

Nuestra estrategia: sin pagar nada

Cogemos el grafo de LangGraph de la sesión 5 y lo instrumentamos con las dos.

- **LangSmith:** dos variables de entorno y ves el árbol completo del grafo
- **LangFuse Cloud:** free tier sin instalar nada, más prompt management y una evaluación pequeña
- **Comparamos las dos vistas** sobre la misma ejecución y cada uno elige
- **Coste total:** **0 EUR** (free tier en las dos; LangFuse además es open source)

Y os lleváis algo que vale en cualquier entrevista: **demostrar con números que vuestro agente funciona**.

IDEA CLAVE

La observabilidad no es un extra que se añade al final si sobra tiempo.

Es lo que convierte un demo que funciona en tu portátil en **una aplicación de producción de verdad**: una que puedes **depurar** cuando falla, **medir** cuando dudas y **mejorar** con datos en vez de con intuiciones.

thePower Tech School

02

Bloque práctico

thePower
Tech School

Punto de partida: observar el agente de la sesión 5

1

El agente, sin tocar

agente.py es el grafo de pedidos de la sesión 5. No sabe nada de LangSmith ni de LangFuse.

2

Tres paquetes nuevos

langfuse, langsmith y python-dotenv (más langchain, que necesita la integración de LangFuse).

3

Dos cuentas gratis

LangSmith y LangFuse Cloud, plan gratuito, sin tarjeta. Sus claves van en un archivo .env.

EL MISMO AGENTE, DOS MIRADAS

Paso 1 LangSmith: dos variables de entorno. **Paso 2** LangFuse: @observe + callback.

Pasos 3-4 prompt management y evaluación en LangFuse. **Paso 5** la misma ejecución en las dos.

Coste total: 0 EUR (modelo local + planes gratuitos).

thePower Tech School

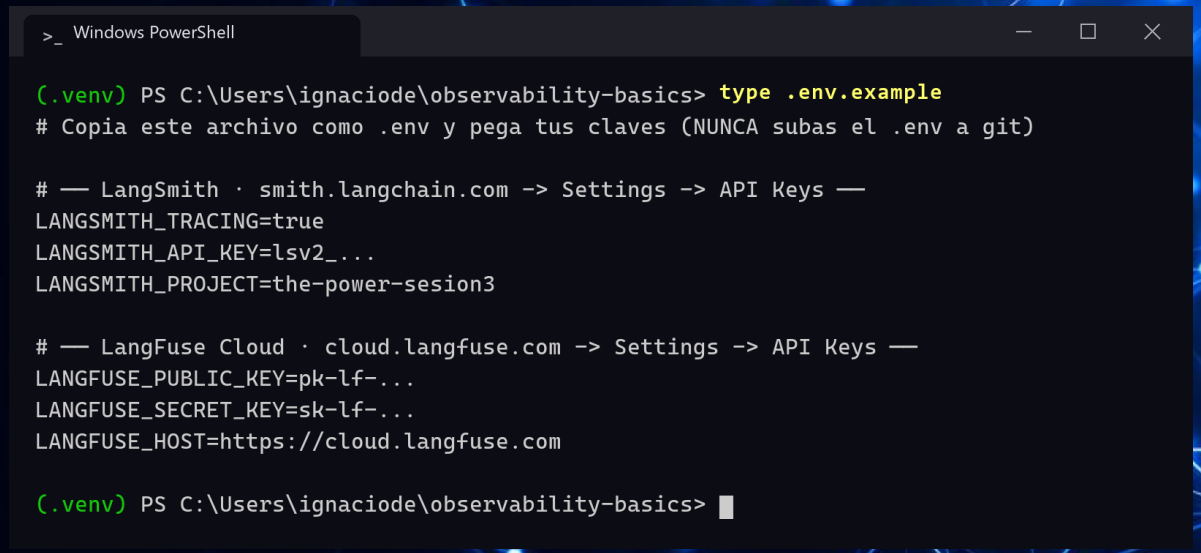
Antes de empezar: cuentas y claves

LANGSMITH · smith.langchain.com

- Sign up (Google, GitHub o email), región US
- Settings -> API Keys -> Create API Key
- Copiar la clave lsv2_... (solo se ve una vez)

LANGFUSE CLOUD · cloud.langfuse.com

- Sign up, crear organización y proyecto
- Settings -> API Keys -> Create new API keys
- Copiar la pareja pk-lf-... y sk-lf-... de la misma ventana



TRES TROPIEZOS REALES AL PREPARAR ESTA DEMO

LANGFUSE_HOST sin https:// · claves de dos parejas distintas · falta el paquete langchain

EL .ENV NO SE COMPARTE NUNCA

En el repo va .env.example con huecos; el .env real con vuestras claves está en .gitignore.

thePower Tech School

Paso 1 · Objetivo

1

OBJETIVO DEL PASO

Ver por dentro el agente sin tocar su código

DE LA TEORÍA

Traces, spans y generations: el árbol de una ejecución. LangSmith se activa con variables de entorno.

LO QUE VAMOS A VER

- La primera traza
- El árbol del grafo
- Los tokens de una llamada

ARCHIVO

- 01_langsmith.py

thePower Tech School

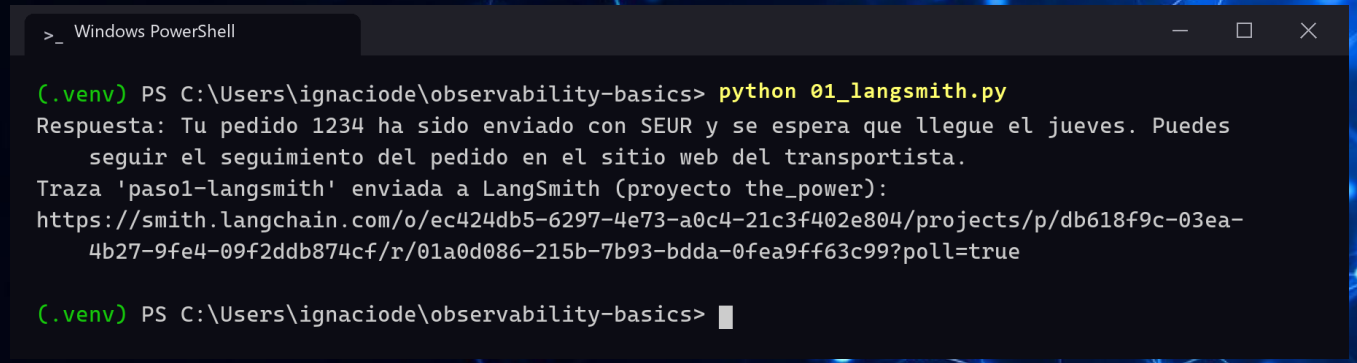
Paso 1 · Instrumentar con LangSmith

QUÉ HACEMOS

- Coger el agente de la sesión 5
- LANGSMITH_TRACING=true + API key en el .env
- Ejecutar el grafo con una pregunta
- Abrir la primera traza en el dashboard

CONCEPTO CLAVE

Dos variables de entorno y el agente de la sesión 5 queda trazado, sin tocar una línea de su código.



FÍJATE EN

la respuesta de siempre y, debajo, el enlace a la traza. El código del agente no ha cambiado.

thePower Tech School

Paso 1 · Anatomía: cero líneas de instrumentación

01_langsmith.py

```python
from dotenv import load_dotenv

# 1) Esto es TODO lo que hace falta: cargar LANGSMITH_TRACING=true y LANGSMITH_API_KEY del .env  (1)
load_dotenv()

from langchain_core.messages import HumanMessage  # noqa: E402

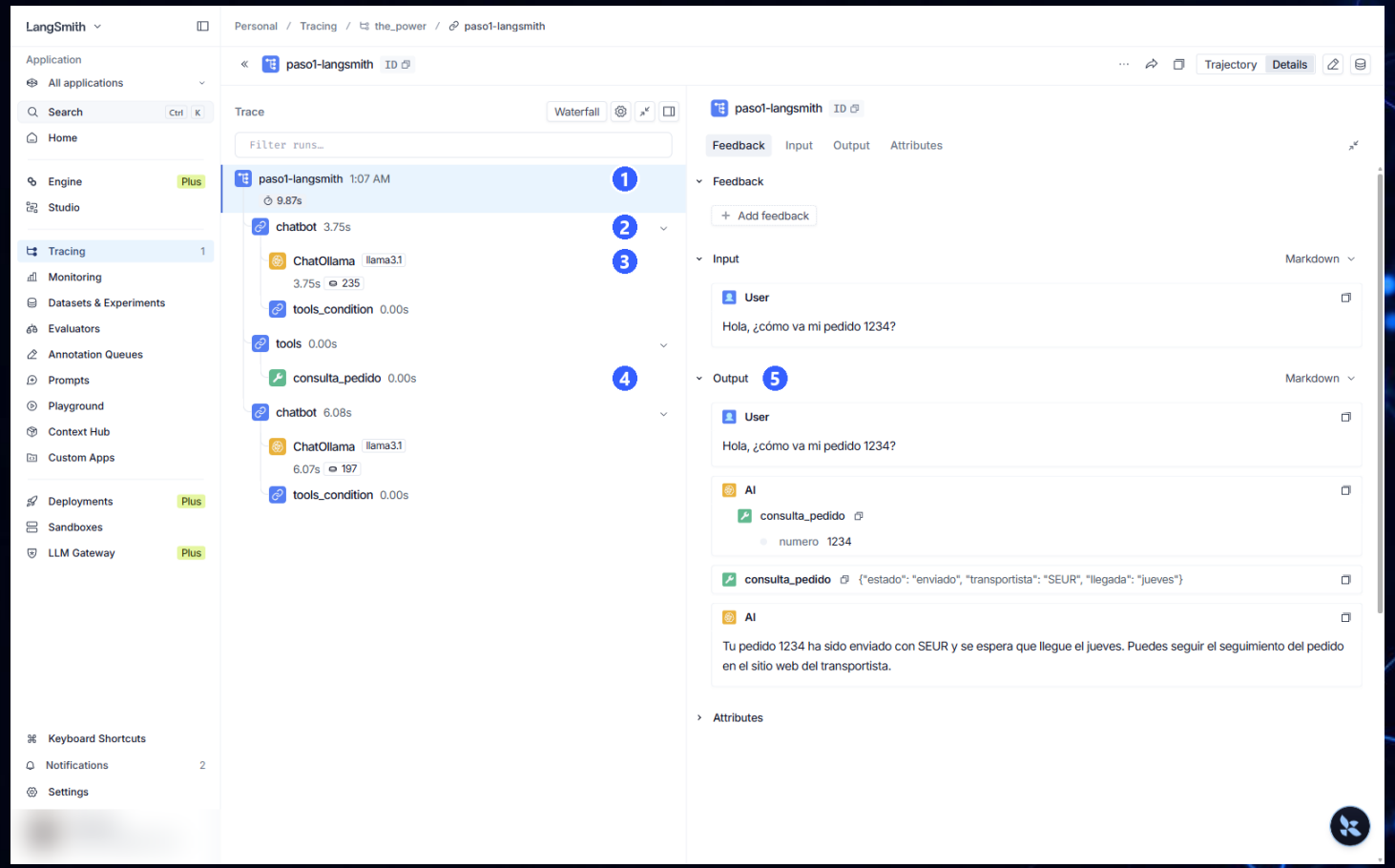
from agente import crear_agente  # noqa: E402  (2)

# 2) El agente de la Sesion 2, tal cual: no importa nada de LangSmith  (3)
grafo = crear_agente()
salida = grafo.invoke({"messages": [HumanMessage("Hola, ¿cómo va mi pedido 1234?")]},
                      config={"run_name": "paso1-langsmith"})
print("Respuesta:", salida["messages"][-1].content)
```

1. **load_dotenv()**
   carga LANGSMITH_TRACING y la API key: con eso LangChain ya traza solo
2. **El agente, importado**
   el mismo grafo de la sesión 5, sin un solo import de LangSmith
3. **invoke de siempre**
   run_name solo pone nombre a la traza; es opcional

thePower Tech School

Paso 1 · La primera traza en LangSmith



1. **La traza**
   una ejecución completa: 9,9 s en total

2. **Spans**
   cada nodo del grafo: chatbot, tools, chatbot

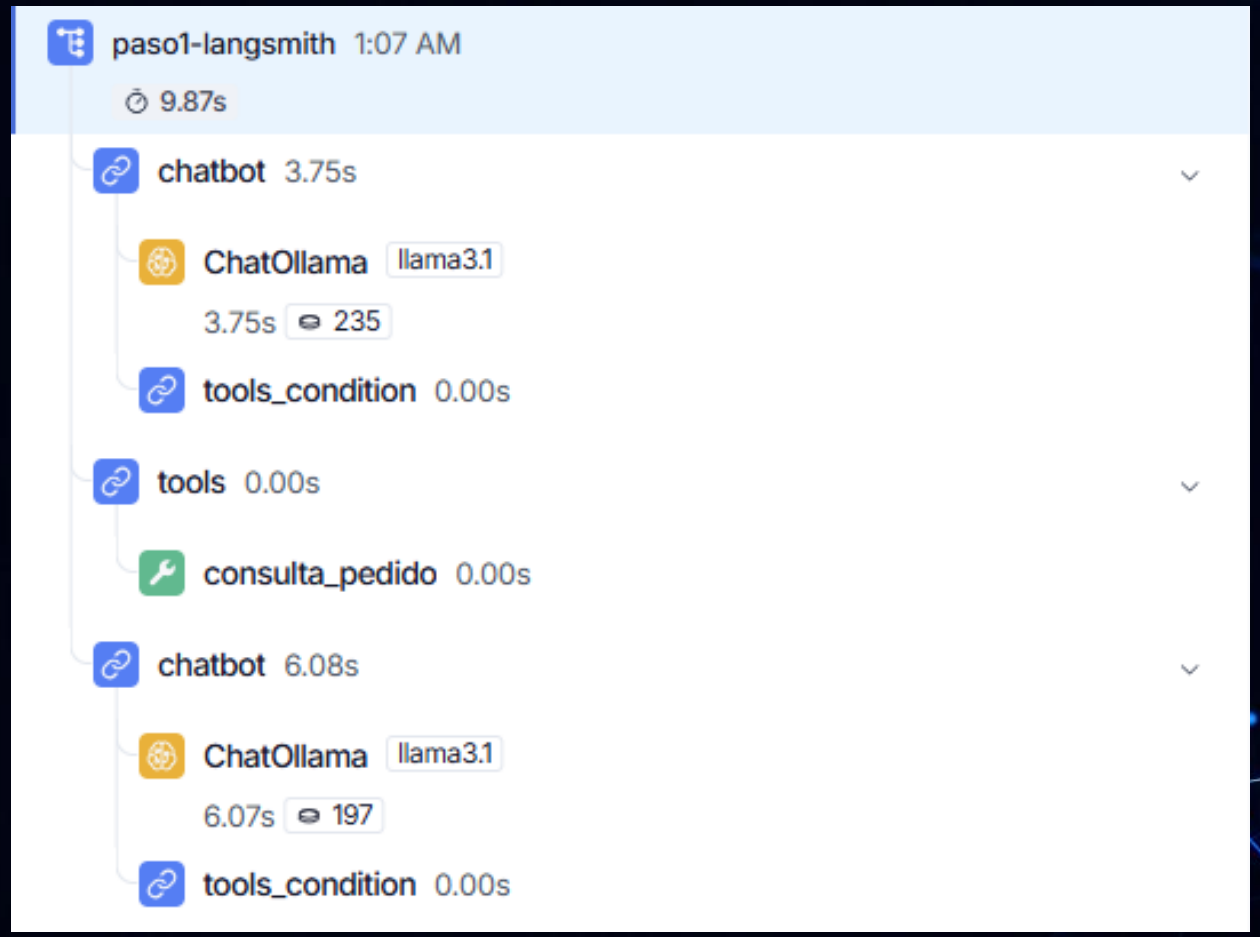
3. **Generations**
   cada llamada a llama3.1, con sus tokens (235)

4. **La tool**
   consulta_pedido: 0,00 s, código nuestro

5. **Entrada y salida**
   la conversación completa, mensaje a mensaje

thePower Tech School

Paso 1 · Zoom: el árbol del grafo



LANGSMITH

El grafo de la sesión 5, visto por dentro

- La traza paso1-langsmith contiene todo: 9,87 s
- chatbot -> tools -> chatbot: el ciclo del agente, tal cual lo dibujamos
- Dentro de cada chatbot, la llamada a ChatOllama con su tiempo y tokens
- tools_condition: la arista condicional también queda registrada

thePower Tech School

Paso 1 · Zoom: entrada y salida



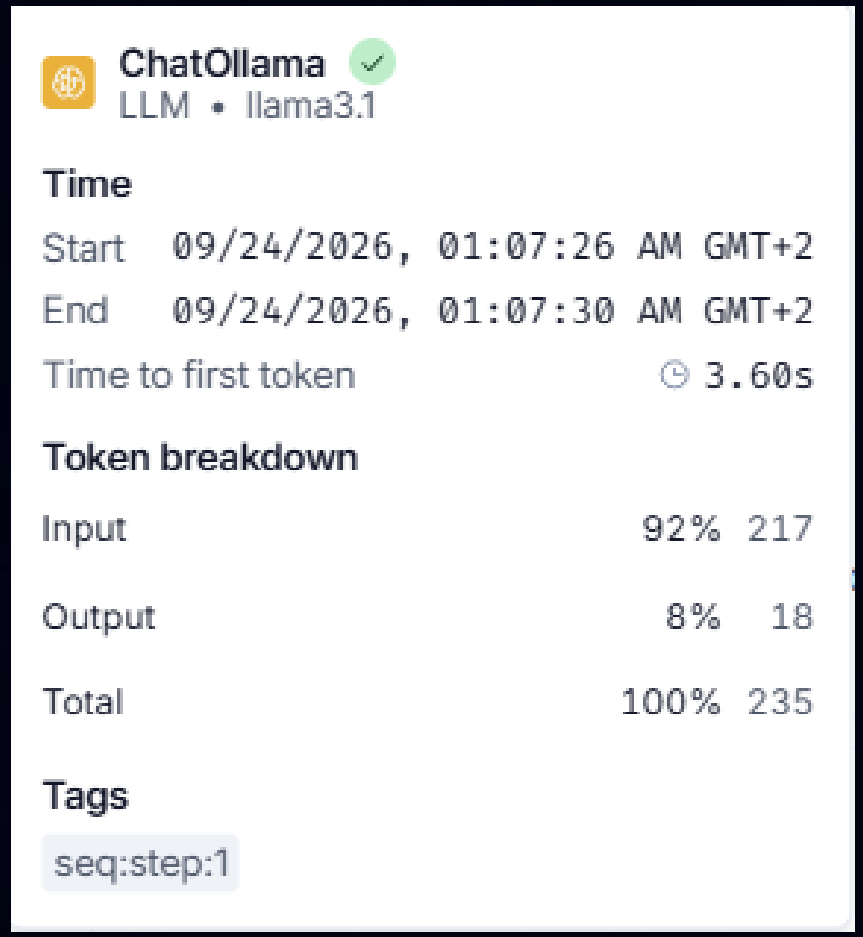
LANGSMITH

La conversación completa

- Input: la pregunta del usuario
- La petición de tool: consulta_pedido con número 1234
- El resultado de la tool, en JSON
- La respuesta final: todo el hilo, sin un solo print

thePower Tech School

Paso 1 · Zoom: dentro de una llamada al modelo



LANGSMITH

Una generation, al detalle

- Tiempo hasta el primer token: 3,6 s (el modelo local piensa despacio)
- Tokens: 217 de entrada y solo 18 de salida
- El 92% de lo que se paga es la ENTRADA: system prompt + historial + definición de tools
- Por eso recortar prompts ahorra más que acortar respuestas

thePower Tech School

Paso 1 · Pregunta a la clase

PREGUNTA A LA CLASE

Si no hemos tocado el código, ¿cómo sabe LangSmith lo que pasa dentro del grafo?

Pista: pensad en quién ejecuta los nodos del grafo.

thePower Tech School

Paso 1 · La respuesta

¿Cómo sabe LangSmith lo que pasa dentro?

- LangChain y LangGraph avisan de cada paso a unos 'callbacks' internos
- Con LANGSMITH_TRACING=true se enciende uno que manda esos avisos a LangSmith
- Por eso es 'gratis' en el mundo LangChain...
- ...y por eso con otras librerías hace falta marcar el código (@traceable), igual que en LangFuse

thePower Tech School

Paso 1 · Lo que te llevas

- ✓ **Tracing sin código** dos variables de entorno y el agente queda observado
- ✓ **El árbol** traza > spans (nodos) > generations (llamadas al modelo)
- ✓ **Las métricas** tokens y latencia por cada llamada
- ✓ **El prompt de entrada pesa** el 92% de los tokens

SIGUIENTE El mismo agente con LangFuse: más trabajo al instrumentar, más control a cambio.

thePower Tech School

Paso 2 · Objetivo

2

OBJETIVO DEL PASO

Instrumentar el mismo agente con LangFuse y leer el coste

DE LA TEORÍA

Qué registra cada llamada al LLM: tokens, coste, latencia, prompt y respuesta.

LO QUE VAMOS A VER

- @observe + CallbackHandler
- Una alucinación cazada
- El coste por llamada

ARCHIVO

- 02_langfuse.py

thePower Tech School

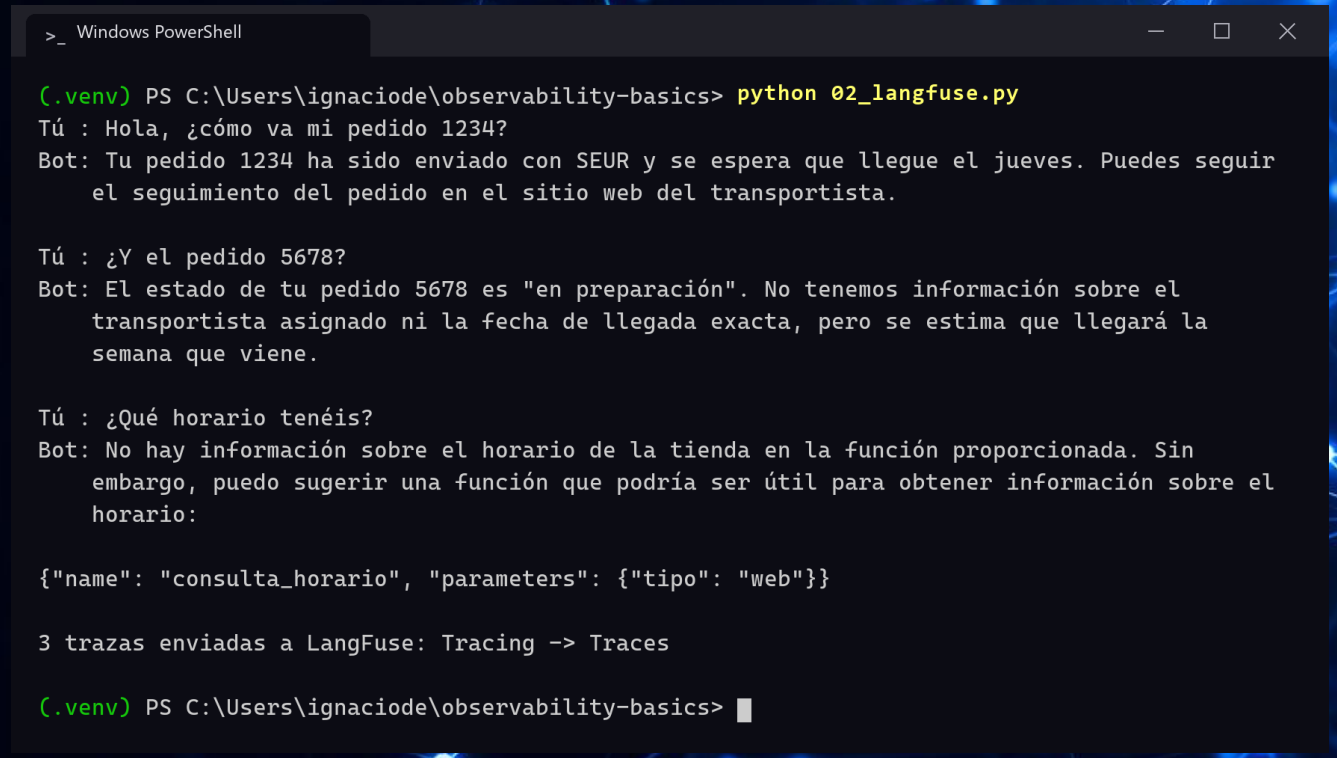
Paso 2 · Instrumentar con LangFuse

QUÉ HACEMOS

- Instalar el SDK de langfuse
- @observe crea la traza
- CallbackHandler añade spans y generations
- Tres preguntas y al dashboard

CONCEPTO CLAVE

Instrumentación explícita: @observe crea la traza y el CallbackHandler añade un span por nodo y una generation por llamada.



FÍJATE EN

la tercera respuesta: a '¿Qué horario tenéis?' el modelo se inventa una tool que no existe.

thePower Tech School

Paso 2 · Zoom: la respuesta sospechosa

```
Tú : ¿Qué horario tenéis?
Bot: No hay información sobre el horario de la tienda en la función proporcionada. Sin
    embargo, puedo sugerir una función que podría ser útil para obtener información sobre el
    horario:

{"name": "consulta_horario", "parameters": {"tipo": "web"}}

3 trazas enviadas a LangFuse: Tracing -> Traces
```

TERMINAL

¿Qué horario tenéis?

- No hay ninguna tool de horarios
- El modelo se inventa una: consulta_horario
- Y la escribe como texto, no como llamada real
- En la terminal parece una rareza; en la traza veremos exactamente qué pasó

thePower Tech School

Paso 2 · Anatomía: instrumentación explícita

02_langfuse.py

```python
load_dotenv()  # LANGFUSE_PUBLIC_KEY, LANGFUSE_SECRET_KEY y LANGFUSE_HOST

from langchain_core.messages import HumanMessage  # noqa: E402
from langfuse import get_client, observe  # noqa: E402
from langfuse.langchain import CallbackHandler  # noqa: E402

from agente import crear_agente  # noqa: E402

langfuse = get_client()
grafo = crear_agente()


# @observe crea la TRAZA (la peticion completa); el CallbackHandler anade dentro
# un SPAN por cada nodo del grafo y una GENERATION por cada llamada al modelo
@observe(name="atender-cliente")
def atender_cliente(pregunta: str) -> str:
    salida = grafo.invoke({"messages": [HumanMessage(pregunta)]},
                          config={"callbacks": [CallbackHandler()]})
    return salida["messages"][-1].content


if __name__ == "__main__":
    for pregunta in ["Hola, ¿cómo va mi pedido 1234?", "¿Y el pedido 5678?", "¿Qué horario tenéis?"]:
        respuesta = atender_cliente(pregunta)
        print(f"Tú : {pregunta}\nBot: {respuesta}\n")
    langfuse.flush()  # enviar todo antes de salir
    print("3 trazas enviadas a LangFuse: Tracing -> Traces")
```

1. **Las claves**
   LANGFUSE_PUBLIC_KEY, SECRET_KEY y HOST desde el .env

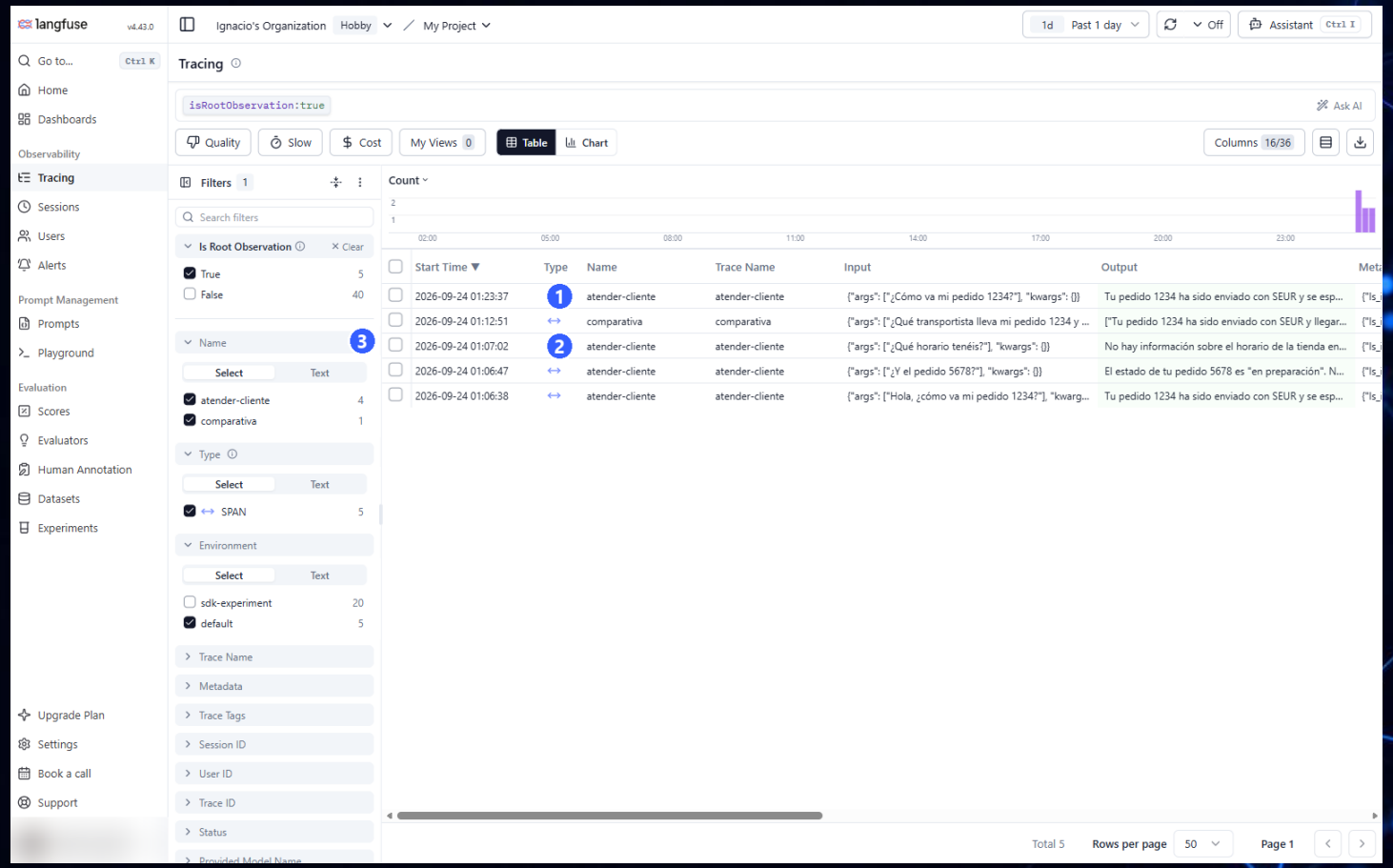
2. **@observe**
   convierte la función en una TRAZA con su nombre

3. **CallbackHandler**
   mete dentro un span por nodo y una generation por llamada

4. **flush()**
   envía lo pendiente antes de salir; sin él se pierden trazas

thePower Tech School

Paso 2 · Las trazas en LangFuse



1. **Una fila, una petición**
   atender-cliente es el nombre que dimos en @observe

2. **La del horario**
   la vamos a abrir: ahí está la tool inventada

3. **Filtros**
   por nombre, tipo, entorno... para encontrar la aguja en el pajar

thePower Tech School

Paso 2 · Zoom: la lista de trazas

Start Time ▼	Type	Name	Trace Name	Input	Output	Meta
2026-09-24 01:23:37	↔	atender-cliente	atender-cliente	{"args": ["¿Cómo va mi pedido 1234?"], "kwargs": {}}	Tu pedido 1234 ha sido enviado con SEUR y se esp...	{"ls_i
2026-09-24 01:12:51	↔	comparativa	comparativa	{"args": ["¿Qué transportista lleva mi pedido 1234 y ...	["Tu pedido 1234 ha sido enviado con SEUR y llegar...	{"ls_i
2026-09-24 01:07:02	↔	atender-cliente	atender-cliente	{"args": ["¿Qué horario tenéis?"], "kwargs": {}}	No hay información sobre el horario de la tienda en...	{"ls_i
2026-09-24 01:06:47	↔	atender-cliente	atender-cliente	{"args": ["¿Y el pedido 5678?"], "kwargs": {}}	El estado de tu pedido 5678 es "en preparación". N...	{"ls_i
2026-09-24 01:06:38	↔	atender-cliente	atender-cliente	{"args": ["Hola, ¿cómo va mi pedido 1234?"], "kwarg...	Tu pedido 1234 ha sido enviado con SEUR y se esp...	{"ls_i

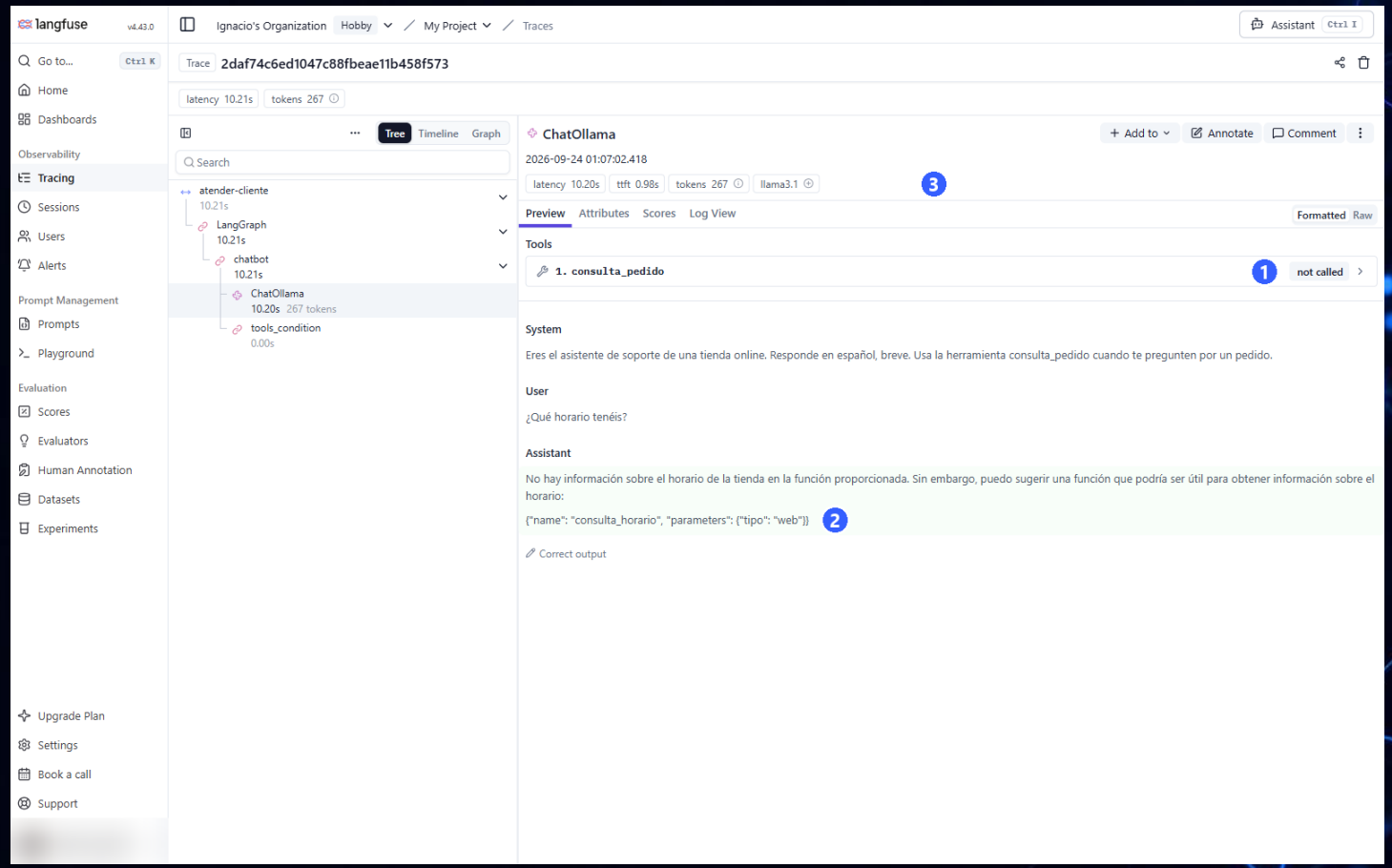
LANGFUSE

Una fila por petición

- Nombre: atender-cliente, el de @observe
- Input y output de cada petición de un vistazo
- Ordenadas por hora: la última arriba
- Clic en una fila y se abre la traza

thePower Tech School

Paso 2 · Caso real: cazar una alucinación



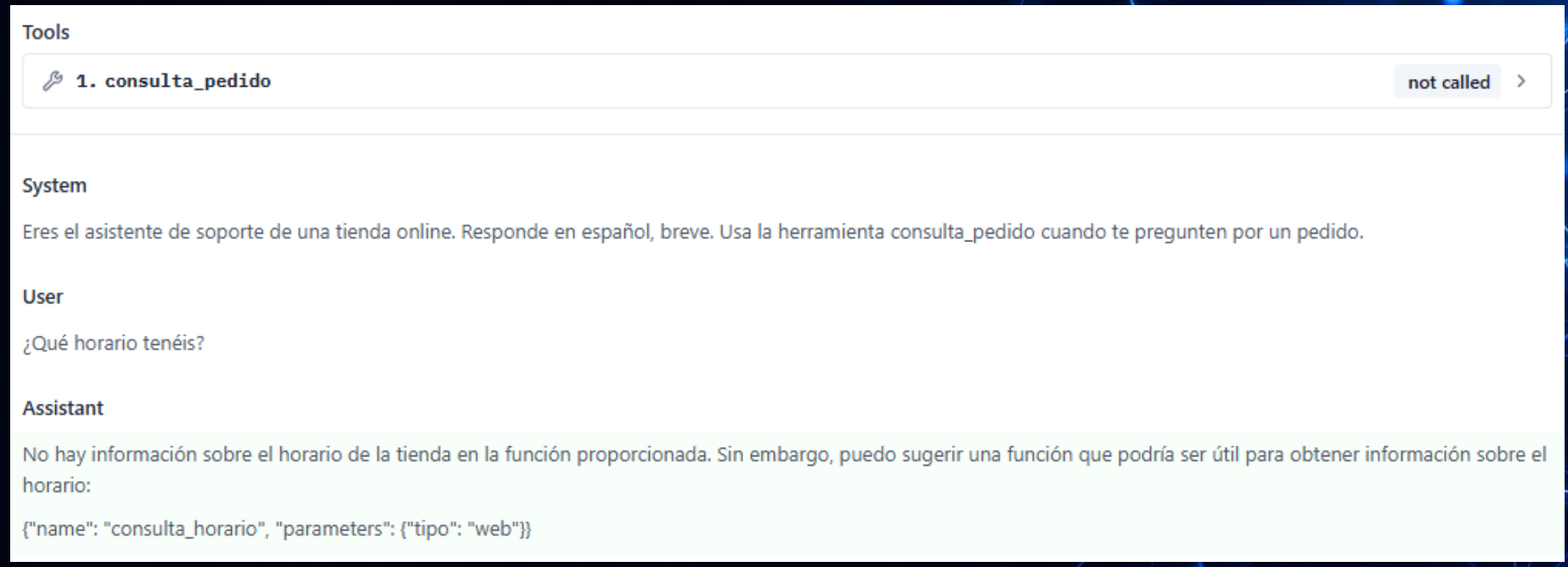
1. **not called**
   la tool consulta_pedido NO se llamó: correcto, no es un pedido

2. **La tool inventada**
   el modelo escribe como texto una llamada a consulta_horario, que no existe

3. **Tokens y latencia**
   267 tokens, 10 s: todo registrado para depurar

thePower Tech School

Paso 2 · Zoom: la tool que no se llamó

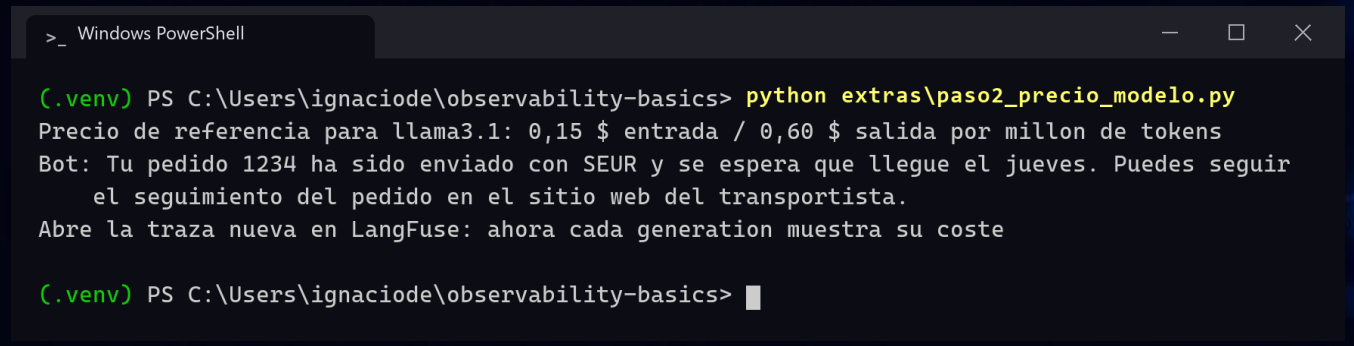


LANGFUSE

La prueba del delito

- consulta_pedido: not called
- El system prompt real que recibió el modelo
- La respuesta con el JSON inventado
- Esto es depurar: ver el prompt, la decisión y la salida juntos

Paso 2 · Variación: ponerle precio al modelo



VARIACIÓN EN VIVO

QUÉ CAMBIAMOS

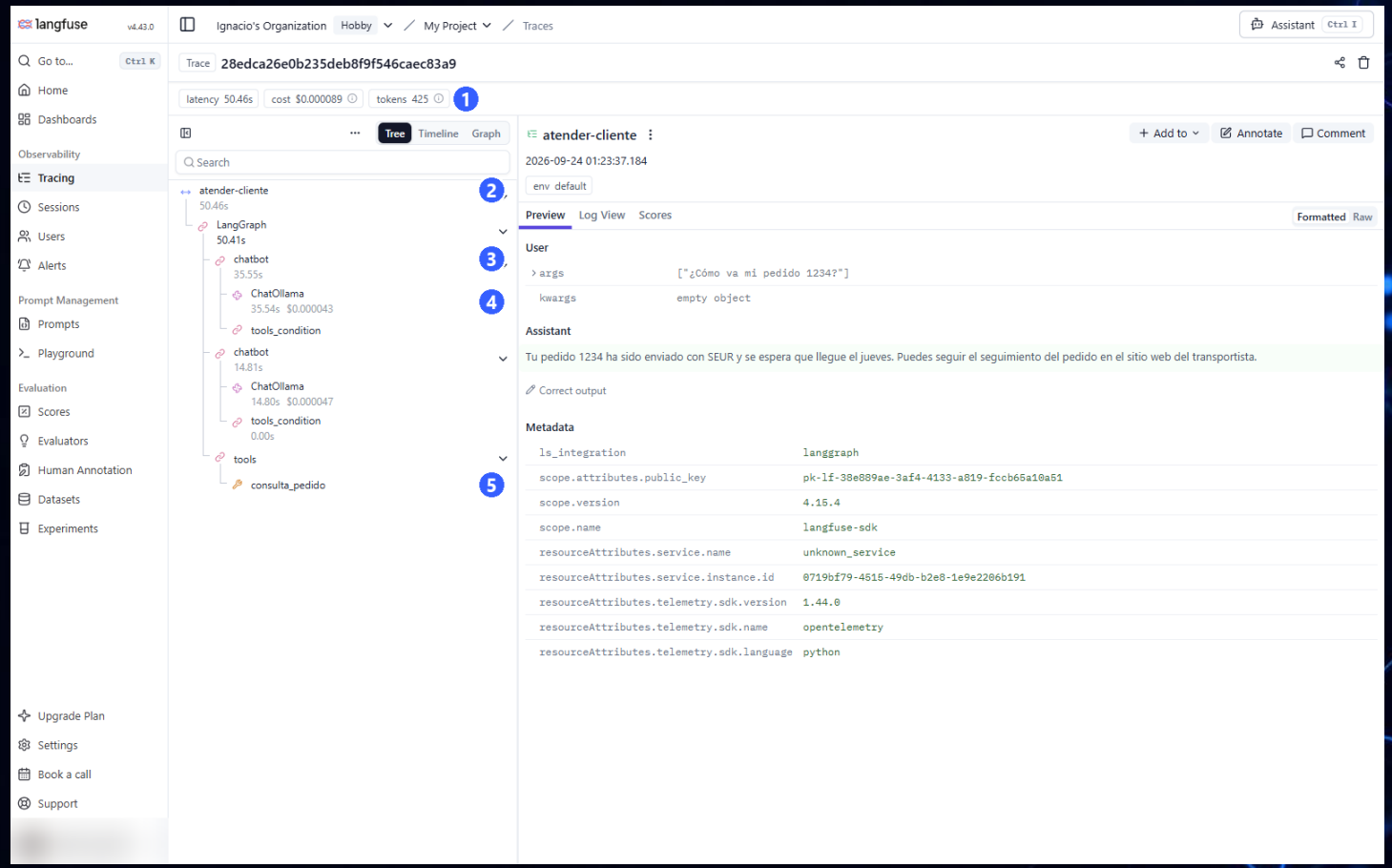
Damos a llama3.1 un precio de referencia en LangFuse y relanzamos una pregunta para ver el coste calculado.

QUÉ DEMUESTRA

Con Ollama el coste real es 0. Dándole un precio de referencia, LangFuse calcula cuánto costaría con una API de pago: la misma traza, ahora con dólares.

thePower Tech School

Paso 2 · El coste, nodo a nodo



1. **Totales**
   latencia 50 s, coste 0,000089 $, 425 tokens

2. **La traza**
   atender-cliente, creada por @observe

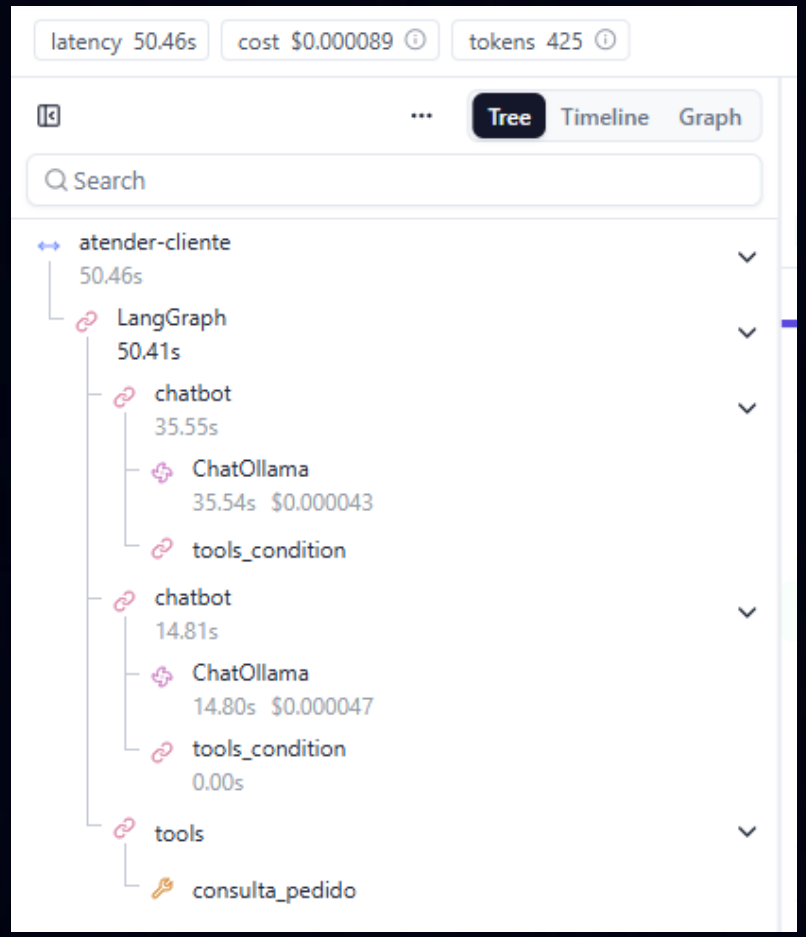
3. **Span del nodo**
   chatbot: el nodo del grafo

4. **Generation con coste**
   cada llamada al modelo, con su precio

5. **La tool**
   consulta_pedido: sin coste, es código nuestro

thePower Tech School

Paso 2 · Zoom: el coste en el árbol



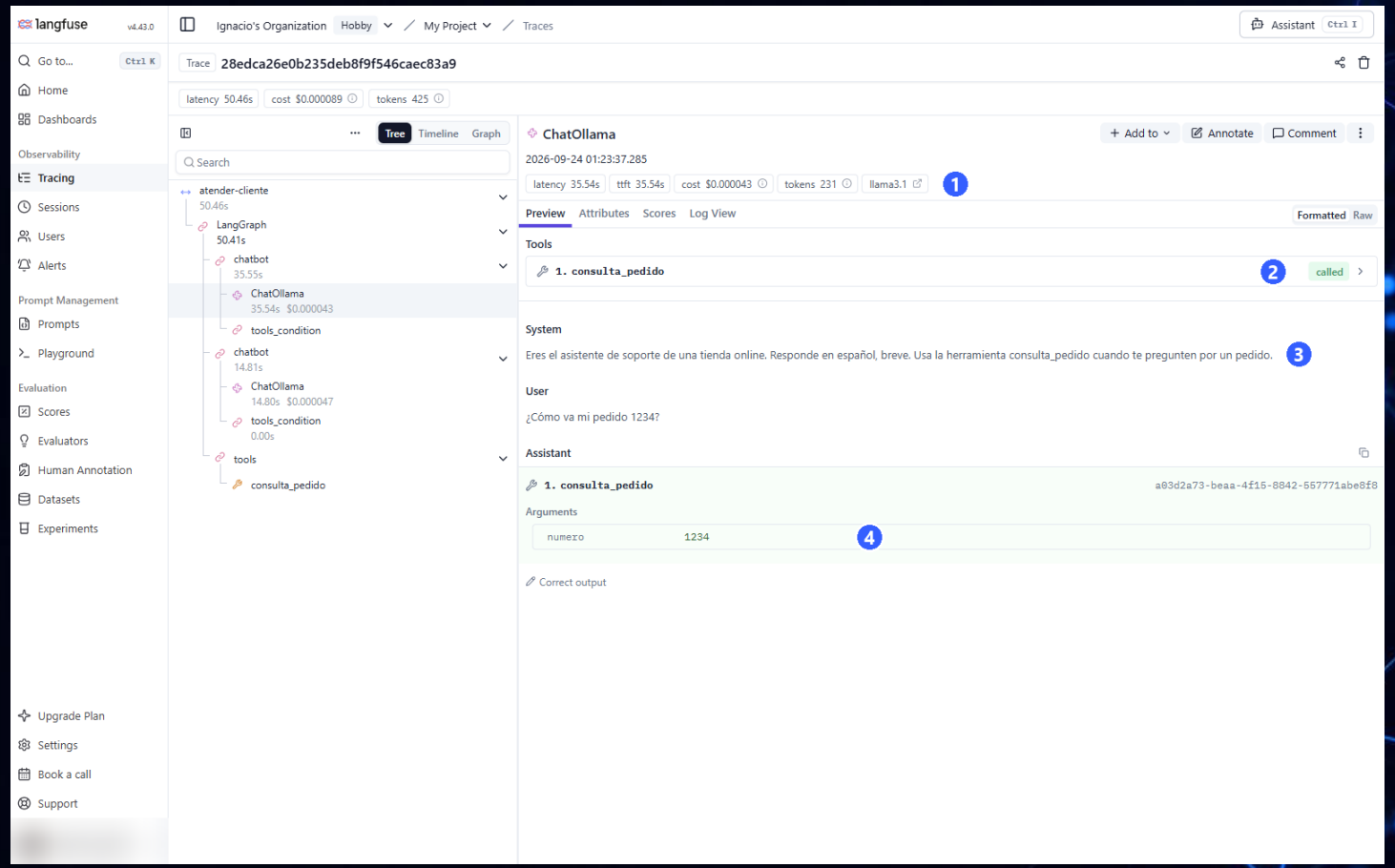
LANGFUSE

Cada generation con su precio

- Arriba, los totales de la traza
- Cada ChatOllama: 0,000043 $ y 0,000047 $
- Los nodos del grafo (spans) no cuestan: solo las llamadas al modelo
- Multiplicad por 10.000 conversaciones al día...

thePower Tech School

Paso 2 · Dentro de una generation



1. **Métricas**
   latencia, primer token, coste, tokens y modelo

2. **Tools disponibles**
   consulta_pedido: called

3. **El prompt REAL**
   el system prompt que de verdad le llegó al modelo

4. **La decisión**
   pide la tool con número = 1234

thePower Tech School

Paso 2 · Zoom: las métricas de una generation



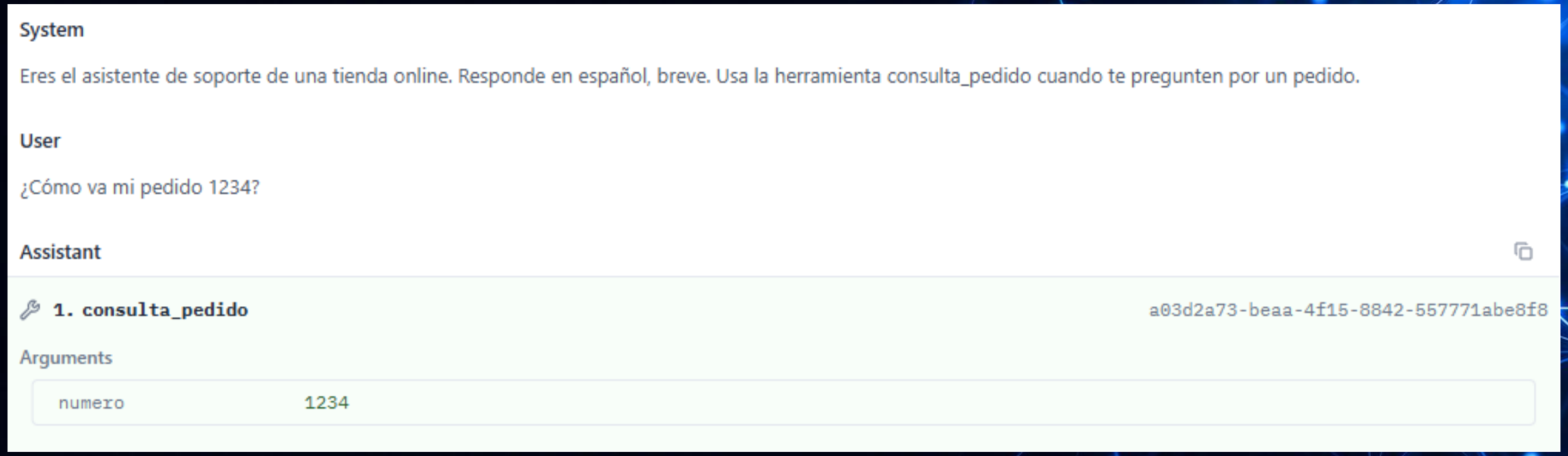
LANGFUSE

La ficha de una llamada

- latency 35,5 s y ttft (tiempo al primer token)
- cost 0,000043 $ y 231 tokens
- El modelo: llama3.1
- Tools: consulta_pedido called

thePower Tech School

Paso 2 · Zoom: el prompt real y la decisión



LANGFUSE

Lo que de verdad vio el modelo

- System: nuestro prompt de soporte
- User: la pregunta
- Assistant: pide consulta_pedido con número 1234
- Cuando algo falla, lo primero es mirar AQUÍ, no el código

Paso 2 · Pregunta a la clase

PREGUNTA A LA CLASE

El modelo se inventó una tool. Sin observabilidad, ¿cuándo os habríais enterado?

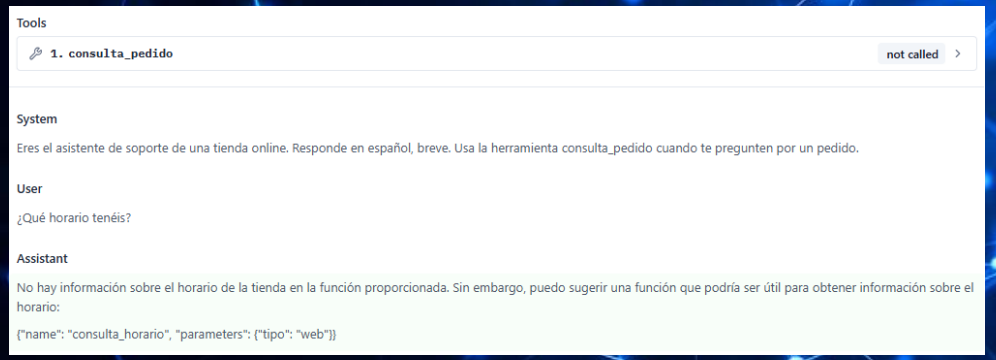
Pensad en producción, con miles de conversaciones al día.

thePower Tech School

Paso 2 · La respuesta

¿Cuándo os habríais enterado?

- Cuando un cliente se quejara... o nunca
- Con trazas: se ve 'not called' y el texto inventado; se puede filtrar y poner alertas
- El arreglo tiene dos caminos: el prompt ('si no tienes el dato, dilo') o una tool real de horarios
- Y después, medir si el arreglo funciona: eso es el paso 4



thePower Tech School

Paso 2 · Lo que te llevas

- ✓ **Instrumentación explícita** @observe crea la traza; el callback, spans y generations
- ✓ **Depurar con datos** prompt real, decisión y respuesta en una pantalla
- ✓ **Coste** LangFuse lo calcula en cuanto el modelo tiene precio
- ✓ **flush()** sin él, las últimas trazas se pierden

SIGUIENTE Sacar el prompt del código: versionarlo y cambiarlo sin desplegar.

thePower Tech School

Paso 3 · Objetivo

3

OBJETIVO DEL PASO

Cambiar el comportamiento del agente sin tocar el código

DE LA TEORÍA

Prompt management: versionar, editar sin desplegar, comparar y hacer rollback.

LO QUE VAMOS A VER

- Dos versiones del prompt
- Cargarlo por nombre
- Promote y rollback

ARCHIVO

- 03a, 03 y 03b

thePower Tech School

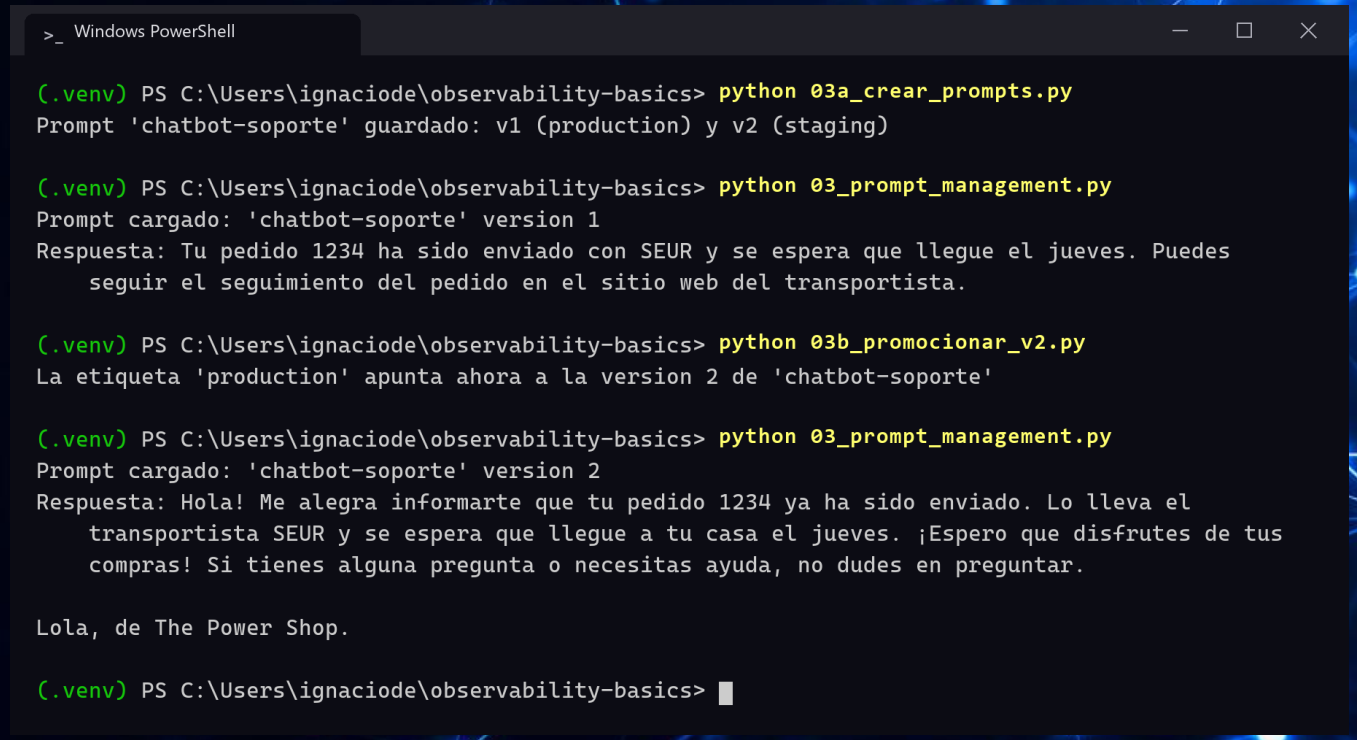
Paso 3 · Prompt management

QUÉ HACEMOS

- Guardar el prompt en LangFuse, en dos versiones
- El código lo pide por nombre y etiqueta
- Mover la etiqueta production a la v2
- Mismo código, otra respuesta

CONCEPTO CLAVE

El prompt vive en LangFuse, versionado. El código lo pide por nombre y etiqueta; la versión se cambia sin desplegar.



FÍJATE EN

las dos ejecuciones de 03_prompt_management.py son el MISMO código: la v2 responde con otro tono y firma.

thePower Tech School

Paso 3 · Anatomía: guardar las versiones

03a_crear_prompts.py

```python
langfuse = get_client()

# Version 1: el prompt de siempre, marcada como "production"
v1 = langfuse.create_prompt(name="chatbot-soporte", prompt=PROMPT_POR_DEFECTO, labels=["production"],
                            commit_message="v1: el prompt de la sesion 2")

# Version 2: mismo prompt con otro tono. Se guarda SIN la etiqueta production
v2 = langfuse.create_prompt(
    name="chatbot-soporte",
    prompt=PROMPT_POR_DEFECTO + " Usa un tono muy cercano, tutea al cliente y firma siempre como "
                                "'Lola, de The Power Shop'.",
    labels=["staging"],
    commit_message="v2: tono cercano con firma",
)
langfuse.flush()
print(f"Prompt 'chatbot-soporte' guardado: v{v1.version} (production) y v{v2.version} (staging)")
```

1

Versión 1

create_prompt con la etiqueta production

2

Versión 2

mismo nombre = nueva versión; etiqueta staging, no production

thePower Tech School

Paso 3 · Anatomía: pedir el prompt por nombre

03_prompt_management.py

```python
langfuse = get_client()

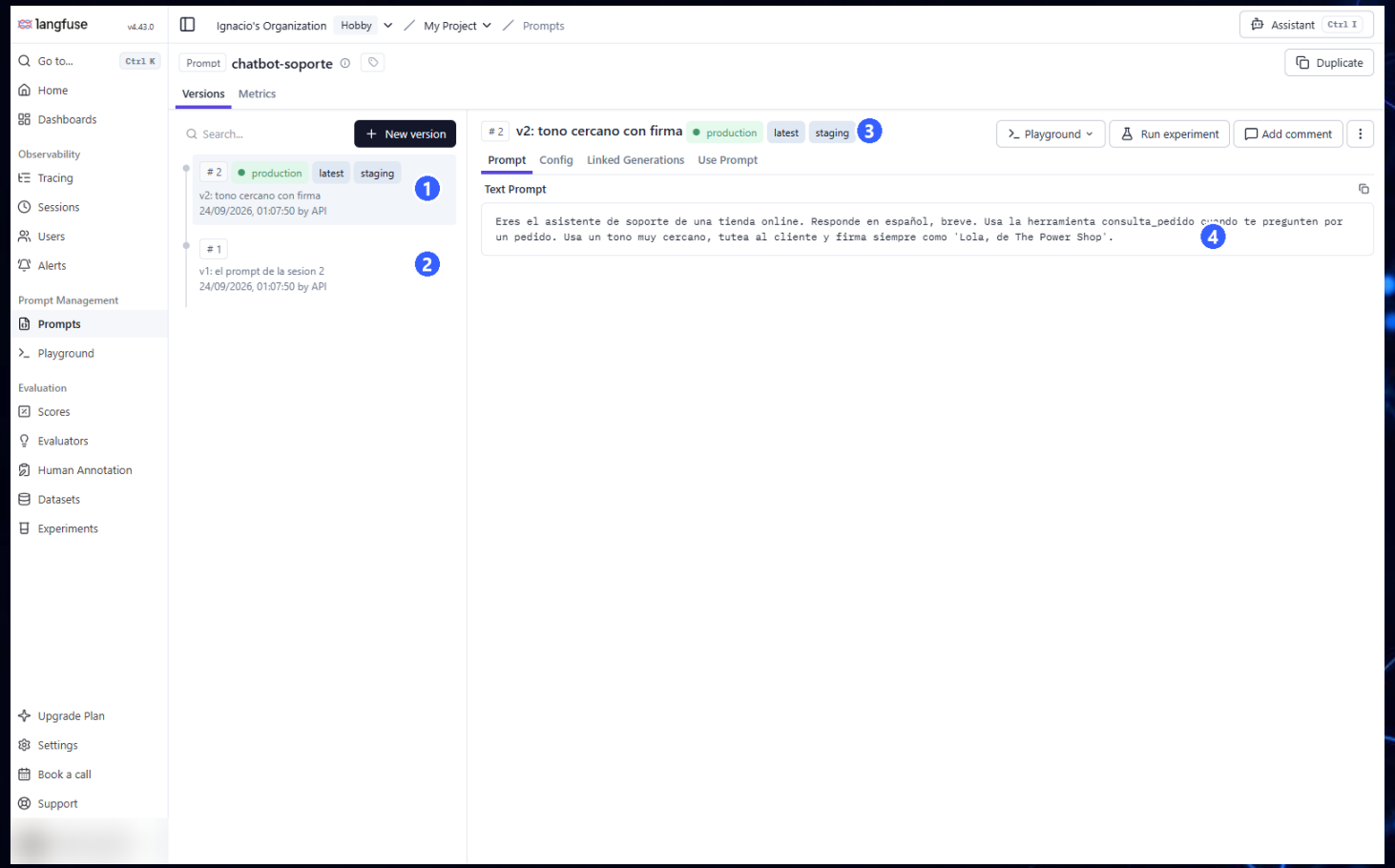
# El prompt ya no esta "quemado" en el codigo: se carga la version con la etiqueta production
prompt = langfuse.get_prompt("chatbot-soporte", label="production", cache_ttl_seconds=0)
print(f"Prompt cargado: '{prompt.name}' version {prompt.version}")

grafo = crear_agente(system_prompt=prompt.compile())
salida = grafo.invoke({"messages": [HumanMessage("Hola, ¿cómo va mi pedido 1234?")]})
print("Respuesta:", salida["messages"][-1].content)
```

1. **get_prompt**
   nombre + etiqueta: la versión la decide LangFuse
2. **Trazabilidad**
   sabemos qué versión se ha usado en cada ejecución
3. **compile()**
   el texto final entra en el agente como system prompt

thePower Tech School

Paso 3 · Las versiones en LangFuse



1. **v2 en producción**
   etiquetas production, latest y staging

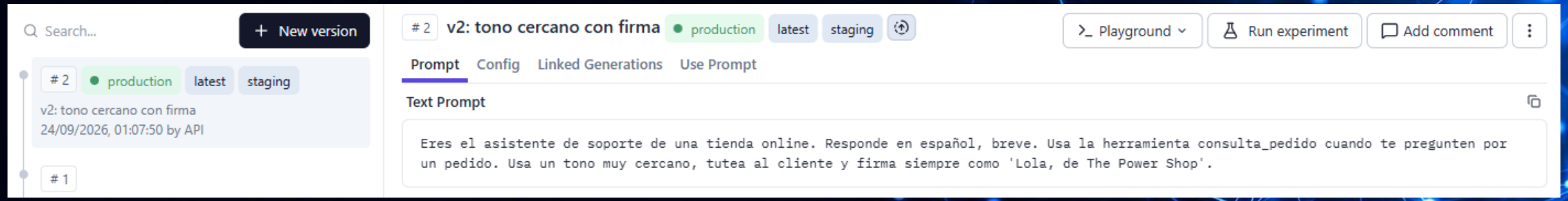
2. **v1**
   sigue ahí: nada se borra, todo es historial

3. **Etiquetas**
   production es solo una etiqueta que se mueve de versión

4. **El texto**
   el prompt completo, editable desde la web

thePower Tech School

Paso 3 · Zoom: versiones y etiquetas



LANGFUSE

Cada cambio es una versión

- #2 lleva production, latest y staging
- #1 sigue guardada con su mensaje de commit
- production es la etiqueta que lee el código
- Moverla es 'desplegar' un prompt

thePower Tech School

Paso 3 · Mismo código, dos respuestas

VERSIÓN 1 · production

"Tu pedido 1234 ha sido enviado con SEUR y se espera que llegue el jueves. Puedes seguir el seguimiento del pedido en el sitio web del transportista."

VERSIÓN 2 · production

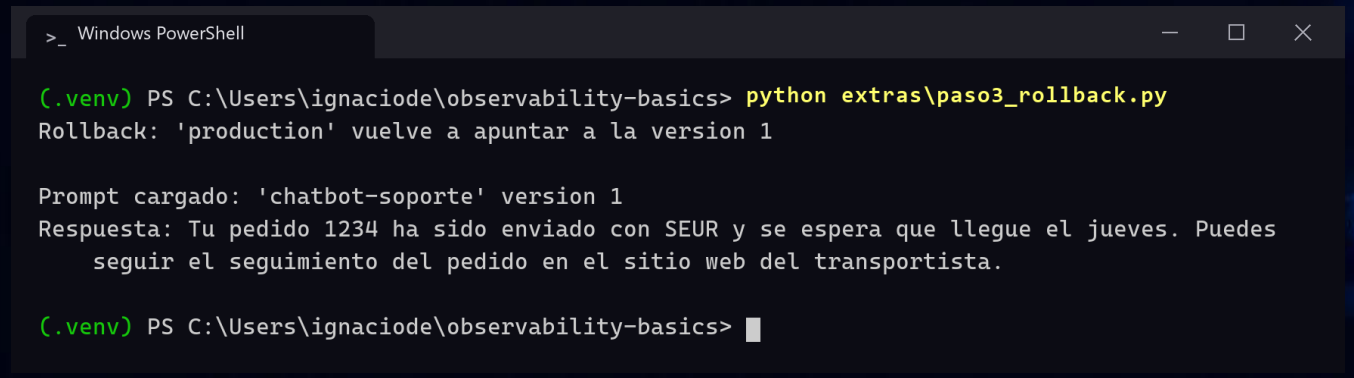
"¡Hola! Me alegra informarte que tu pedido 1234 ya ha sido enviado. Lo lleva el transportista SEUR y se espera que llegue a tu casa el jueves. ¡Espero que disfrutes de tus compras! (...) Lola, de The Power Shop."

Entre una y otra no se ha tocado ni desplegado código: solo se ha movido la etiqueta production de la v1 a la v2.

En un equipo real, quien ajusta el tono puede no ser quien programa.

thePower Tech School

Paso 3 · Variación: rollback en un segundo



VARIACIÓN EN VIVO

QUÉ CAMBIAMOS

Devolvemos la etiqueta production a la versión 1 y ejecutamos otra vez el mismo script del paso 3.

QUÉ DEMUESTRA

Si la v2 da problemas en producción, volver atrás es mover una etiqueta: el mismo script carga la versión 1 al instante, sin desplegar nada.

thePower Tech School

Paso 3 · Zoom: después del rollback



LANGFUSE

production vuelve a la v1

- La etiqueta production ha saltado a la #1
- La #2 no se borra: sigue lista para volver
- Todo queda en el historial: quién y cuándo
- Rollback en segundos, sin tocar el repositorio

thePower Tech School

Paso 3 · Pregunta a la clase

PREGUNTA A LA CLASE

Si igualmente hay que probar el prompt, ¿qué ganamos sacándolo del código?

Pensad en quién suele pedir los cambios de tono.

thePower Tech School

Paso 3 · La respuesta

¿Qué ganamos?

- Cambiar sin desplegar: segundos, no un ciclo de release
- Historial y rollback: cada versión queda guardada
- Que alguien de negocio ajuste el tono sin tocar código
- Y medir cada versión con experimentos (paso 4). Riesgo: cambiar production sin probar; para eso está staging

thePower Tech School

Paso 3 · Lo que te llevas

- ✓ **Prompt versionado** create_prompt con el mismo nombre = nueva versión
- ✓ **Por nombre y etiqueta** get_prompt(nombre, label='production')
- ✓ **Promote** mover production cambia el comportamiento al instante
- ✓ **Rollback** volver atrás es mover la etiqueta

SIGUIENTE ¿La v2 es mejor que la v1? No lo opinamos: lo medimos.

thePower Tech School

Paso 4 · Objetivo

4

OBJETIVO DEL PASO

Medir en vez de opinar

DE LA TEORÍA

Evaluación: datasets, evaluadores por reglas y LLM-as-a-judge, métricas comparables.

LO QUE VAMOS A VER

- Un dataset de 5 preguntas
- Regla contra juez
- Comparar experimentos

ARCHIVO

- 04_evaluacion.py

thePower Tech School

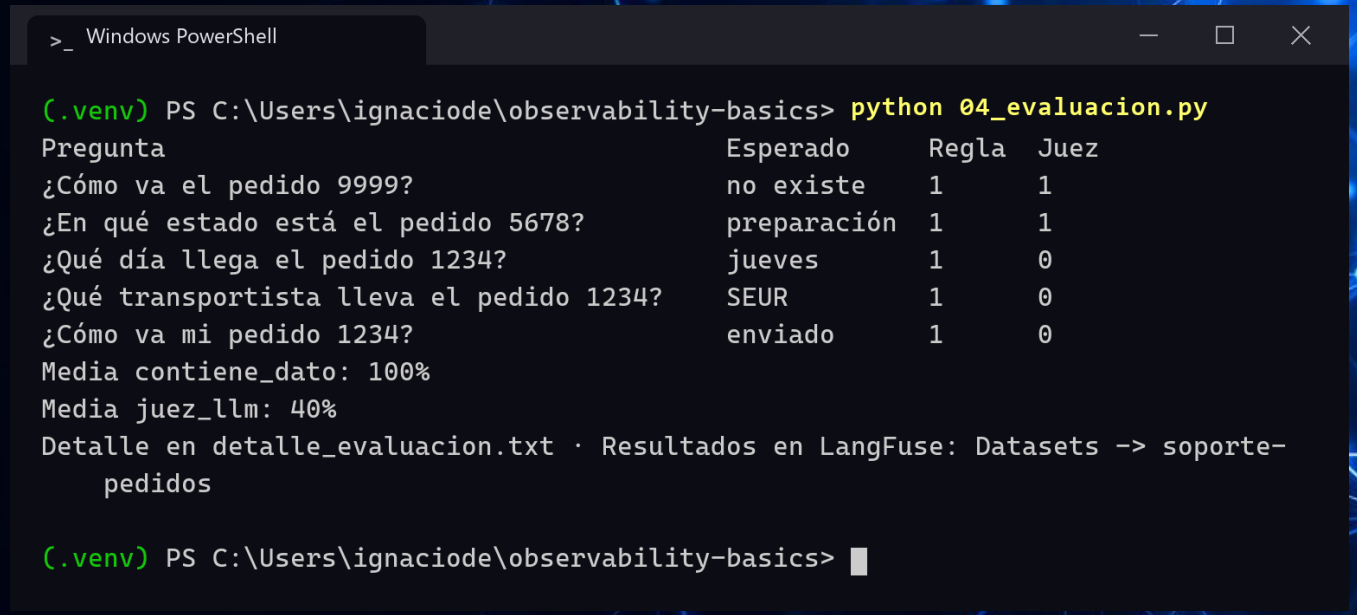
Paso 4 · Evaluación

QUÉ HACEMOS

- Dataset de 5 preguntas con el dato esperado
- El agente sobre todo el dataset
- Evaluador por regla + juez LLM
- Tabla de resultados

CONCEPTO CLAVE

Dataset + tarea + evaluadores = métricas comparables. Y también hay que revisar al evaluador.



FÍJATE EN

la regla dice 100%, el juez solo 40%. ¿Quién tiene razón? Lo vemos en dos slides.

thePower Tech School

Paso 4 · Zoom: la tabla de resultados

```
(.venv) PS C:\Users\ignaciode\observability-basics> python 04_evaluacion.py
Pregunta                                        Esperado     Regla  Juez
¿Cómo va el pedido 9999?                        no existe    1      1
¿En qué estado está el pedido 5678?             preparación  1      1
¿Qué día llega el pedido 1234?                  jueves       1      0
¿Qué transportista lleva el pedido 1234?        SEUR         1      0
¿Cómo va mi pedido 1234?                        enviado      1      0
Media contiene_dato: 100%
Media juez llm: 40%
```

TERMINAL

Dos evaluadores, dos opiniones

- Regla (contiene_dato): 5 de 5
- Juez LLM: solo 2 de 5
- Discrepan justo en las preguntas del pedido 1234
- ¿Está mal el agente o está mal el juez?

thePower Tech School

Paso 4 · Anatomía: dataset, tarea y evaluadores

04_evaluacion.py

```python
# 1) DATASET: 5 preguntas con el dato que la respuesta DEBE contener
EJEMPLOS = [
    ("¿Cómo va mi pedido 1234?", "enviado"),
    ("¿Qué transportista lleva el pedido 1234?", "SEUR"),
    ("¿Qué día llega el pedido 1234?", "jueves"),
    ("¿En qué estado está el pedido 5678?", "preparación"),
    ("¿Cómo va el pedido 9999?", "no existe"),
]
try:
    langfuse.get_dataset(DATASET)
except Exception:
    langfuse.create_dataset(name=DATASET, description="Preguntas de soporte con el dato esperado")
    for pregunta, esperado in EJEMPLOS:
        langfuse.create_dataset_item(dataset_name=DATASET, input=pregunta, expected_output=esperado)
    langfuse.flush()

grafo = crear_agente()
juez = ChatOllama(model="llama3.1", temperature=0)


# 2) TAREA: lo que se ejecuta para cada ejemplo del dataset (nuestro agente)
def tarea(*, item, **kwargs):
    salida = grafo.invoke({"messages": [HumanMessage(item.input)]}, config={"callbacks": [CallbackHandler()]})
    return salida["messages"][-1].content

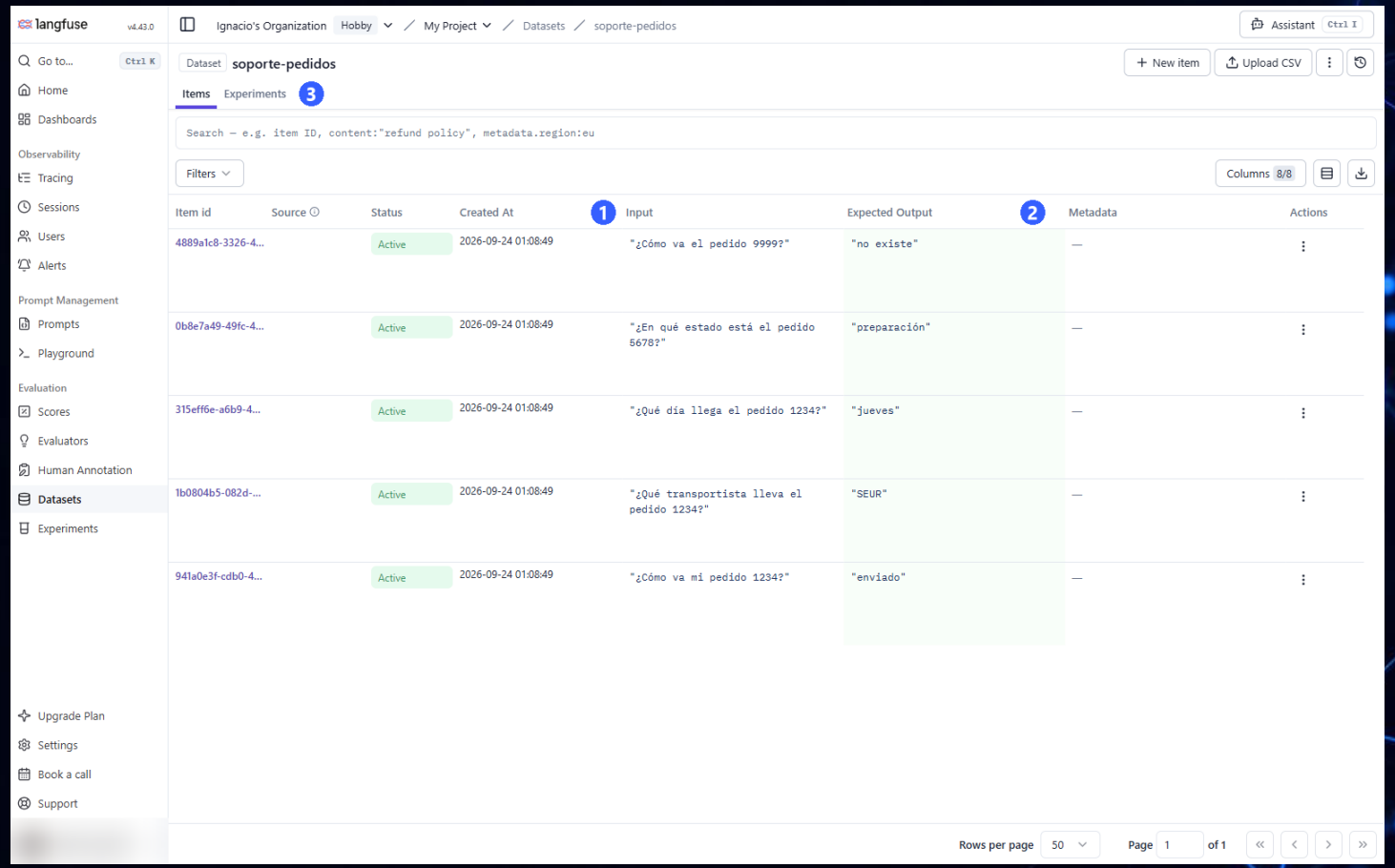

# 3) EVALUADOR POR REGLA: ¿la respuesta contiene el dato esperado?
def contiene_dato(*, output, expected_output, **kwargs):
    ok = expected_output.lower() in output.lower()
    return Evaluation(name="contiene_dato", value=1.0 if ok else 0.0,
                      comment=f"busca '{expected_output}'")


# 4) EVALUADOR LLM-AS-A-JUDGE: otro modelo juzga si la respuesta es correcta
def juez_llm(*, input, output, expected_output, **kwargs):
    veredicto = juez.invoke(
        f"Pregunta de un cliente: {input}\nDato correcto: {expected_output}\nRespuesta del asistente: {output}\n\n"
        "¿La respuesta es correcta y coherente con el dato correcto? Contesta SOLO 'SI' o 'NO'."
    ).content.strip().upper()
    return Evaluation(name="juez_llm", value=1.0 if veredicto.startswith("SI") else 0.0, comment=veredicto)
```

1. **Dataset**
   5 preguntas con el dato que la respuesta debe contener
2. **Tarea**
   lo que se ejecuta por ejemplo: nuestro agente
3. **Evaluador por regla**
   ¿Contiene el dato? 1 o 0. Barato y predecible
4. **LLM-as-a-judge**
   otro modelo decide si la respuesta es correcta

thePower Tech School

Paso 4 · El dataset en LangFuse



1. **Input**
   la pregunta del cliente

2. **Expected output**
   el dato que la respuesta debe contener

3. **Experiments**
   cada ejecución del agente sobre el dataset

thePower Tech School

Paso 4 · Zoom: los ejemplos del dataset

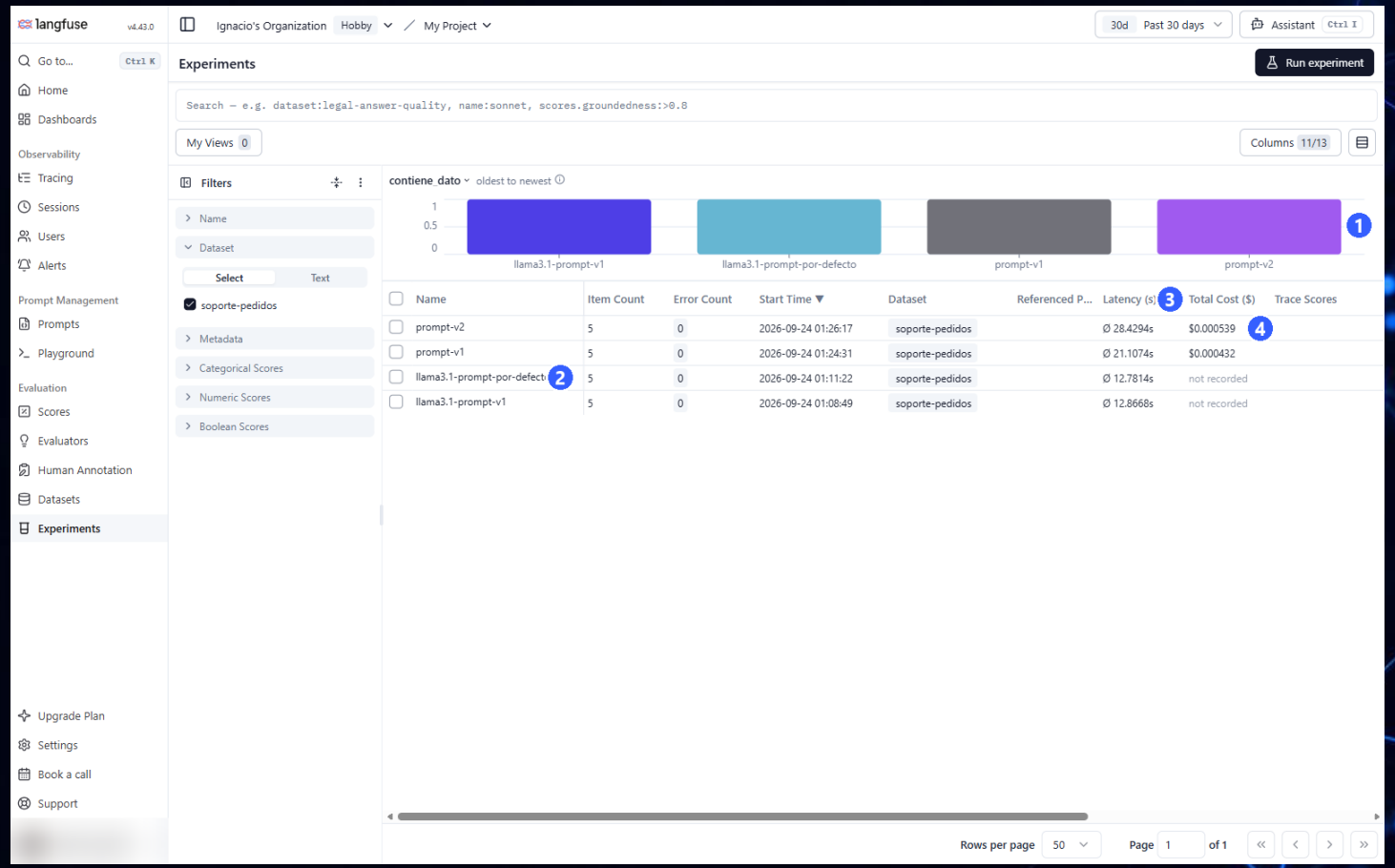
Item id	Source	Status	Created At	Input	Expected Output
4889a1c8-3326-4...		Active	2026-09-24 01:08:49	"¿Cómo va el pedido 9999?"	"no existe"
0b8e7a49-49fc-4...		Active	2026-09-24 01:08:49	"¿En qué estado está el pedido 5678?"	"preparación"
315eff6e-a6b9-4...		Active	2026-09-24 01:08:49	"¿Qué día llega el pedido 1234?"	"jueves"
1b0804b5-082d-...		Active	2026-09-24 01:08:49	"¿Qué transportista lleva el pedido 1234?"	"SEUR"
941a0e3f-cdb0-4...		Active	2026-09-24 01:08:49	"¿Cómo va mi pedido 1234?"	"enviado"

LANGFUSE

Input y expected output

- Cinco preguntas reales de soporte
- El dato que la respuesta debe contener
- Incluye un caso límite: el pedido 9999, que no existe
- Un dataset pequeño pero bien pensado vale más que uno enorme

Paso 4 · Los experimentos



1. **La métrica**
   contiene_dato por experimento, de un vistazo

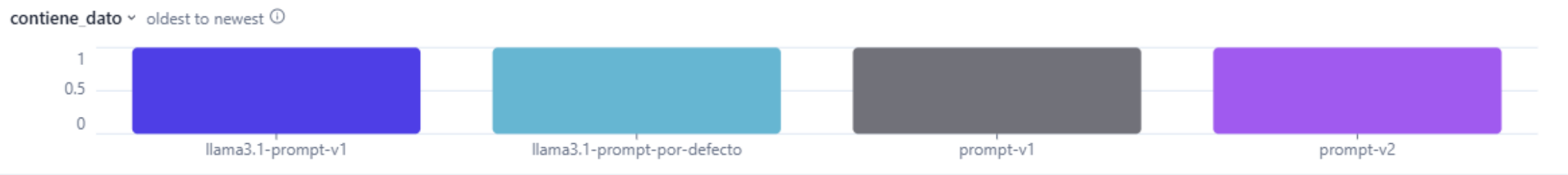
2. **Cada ejecución**
   4 experimentos sobre el mismo dataset

3. **Latencia media**
   de 13 a 28 s según el prompt

4. **Coste**
   aparece desde que el modelo tiene precio

thePower Tech School

Paso 4 · Zoom: la métrica por experimento



Name	Item Count	Error Count	Start Time ▼	Dataset	Referenced P...	Latency (s)	Total Cost ($)	Trace Scores
prompt-v2	5	0	2026-09-24 01:26:17	soporte-pedidos		Ø 28.4294s	$0.000539	
prompt-v1	5	0	2026-09-24 01:24:31	soporte-pedidos		Ø 21.1074s	$0.000432	
llama3.1-prompt-por-defecto	5	0	2026-09-24 01:11:22	soporte-pedidos		Ø 12.7814s	not recorded	
llama3.1-prompt-v1	5	0	2026-09-24 01:08:49	soporte-pedidos		Ø 12.8668s	not recorded	

LANGFUSE

contiene_dato en cada ejecución

- Una barra por experimento
- Todas a 1: la regla siempre encuentra el dato
- Con métricas así se ve de un vistazo si un cambio empeora algo
- Es el termómetro para cada cambio de prompt o de modelo

thePower Tech School

Paso 4 · ¿Quién evalúa al evaluador?

```
>_ Windows PowerShell

(.venv) PS C:\Users\ignaciode\observability-basics> type detalle_evaluacion.txt
PREGUNTA : ¿Cómo va el pedido 9999?
ESPERADO : no existe
RESPUESTA: Lo siento, pero parece que el pedido 9999 no existe en nuestro sistema. ¿Podrías
    proporcionar más información o el número del pedido correcto para que pueda ayudarte mejor?
  contiene_dato: 1 (busca 'no existe')
  juez_llm: 1 (SI)

PREGUNTA : ¿En qué estado está el pedido 5678?
ESPERADO : preparación
RESPUESTA: El estado de tu pedido 5678 es "en preparación". No tenemos información sobre el
    transportista asignado ni sobre la fecha de llegada. Estamos trabajando para que tu pedido
    llegue lo antes posible.
  contiene_dato: 1 (busca 'preparación')
  juez_llm: 1 (SI)

PREGUNTA : ¿Qué día llega el pedido 1234?
ESPERADO : jueves
RESPUESTA: El pedido 1234 se enviará con SEUR y llegará el jueves.
  contiene_dato: 1 (busca 'jueves')
  juez_llm: 0 (NO)

PREGUNTA : ¿Qué transportista lleva el pedido 1234?
ESPERADO : SEUR
RESPUESTA: El transportista que lleva el pedido 1234 es SEUR. Se espera que llegue el jueves.
  contiene_dato: 1 (busca 'SEUR')
  juez_llm: 0 (NO)

PREGUNTA : ¿Cómo va mi pedido 1234?
ESPERADO : enviado
RESPUESTA: Tu pedido 1234 ha sido enviado con SEUR y se espera que llegue el jueves. Puedes seguir
    el seguimiento del pedido en el sitio web del transportista.
  contiene_dato: 1 (busca 'enviado')
  juez_llm: 0 (NO)

(.venv) PS C:\Users\ignaciode\observability-basics>
```

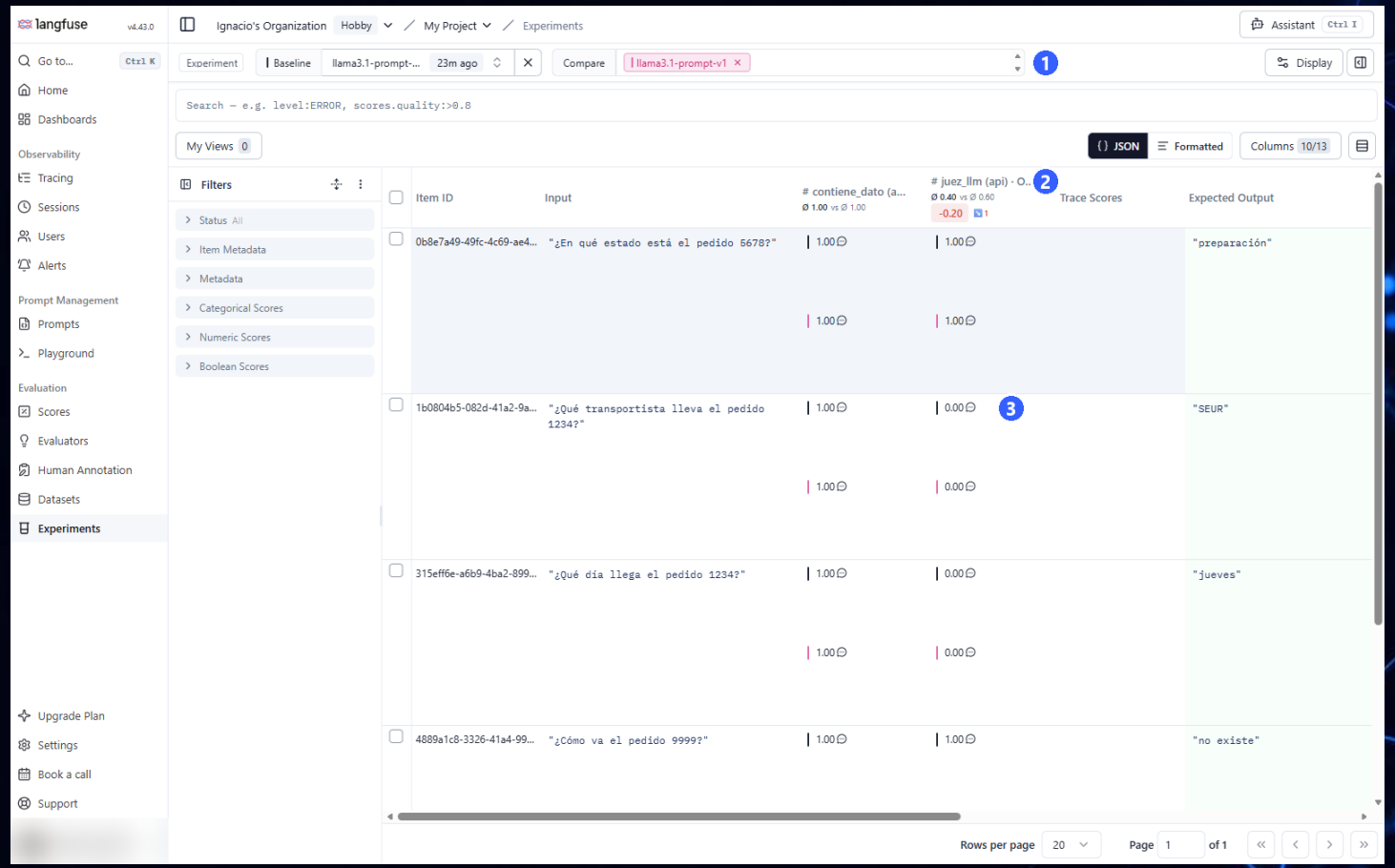
DETALLE_EVALUACION.TXT

El juez se equivoca

- Las 5 respuestas del agente son correctas: todas contienen el dato
- El juez (llama3.1 8B) dice NO a 3 de ellas cuando añaden información extra
- Y no es estable: 60% en una ejecución, 40% en otra
- Regla: el juez debe ser más potente que el evaluado y hay que calibrarlo con revisión humana

thePower Tech School

Paso 4 · El juez, en la comparativa



1. **Baseline vs compare**
   dos ejecuciones con el MISMO prompt

2. **juez_llm**
   0,40 frente a 0,60: -0,20 sin cambiar nada

3. **Pregunta a pregunta**
   el transportista: el juez dice 0 en las dos

thePower Tech School

Paso 4 · Zoom: el juez no es estable

Item ID	Input	# contiene_dato (a... Ø 1.00 vs Ø 1.00	# juez_llm (api) · O... Ø 0.40 vs Ø 0.60 -0.20 ↘1	Trace Scores	E
0b8e7a49-49fc-4c69-ae4...	"¿En qué estado está el pedido 5678?"	1.00	1.00		
		1.00	1.00		
1b0804b5-082d-41a2-9a...	"¿Qué transportista lleva el pedido 1234?"	1.00	0.00		
		1.00	0.00		

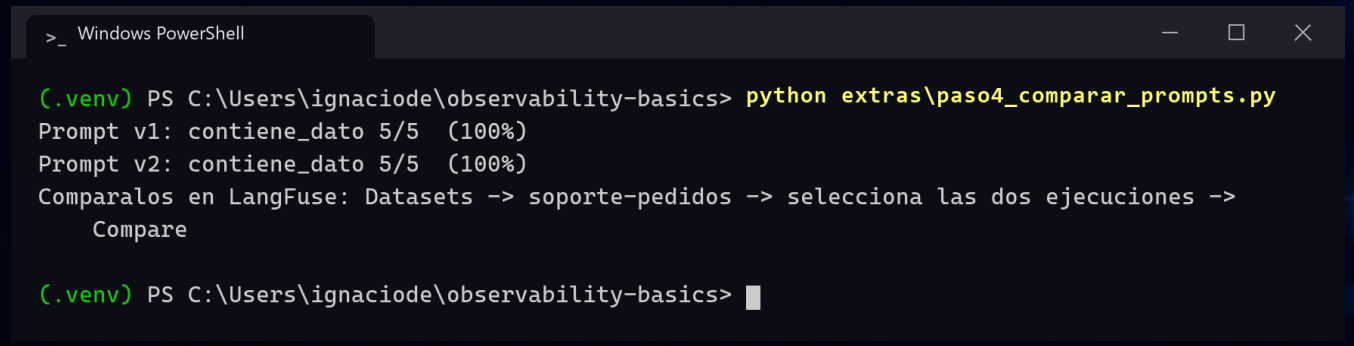
LANGFUSE

Mismo prompt, distinto veredicto

- juez_llm: media 0,40 contra 0,60
- -0,20 sin haber cambiado nada
- Pregunta a pregunta: 1,00 / 0,00 según la ejecución
- Un juez inestable da métricas que no significan nada

thePower Tech School

Paso 4 · Variación: ¿es mejor la v2?



VARIACIÓN EN VIVO

QUÉ CAMBIAMOS

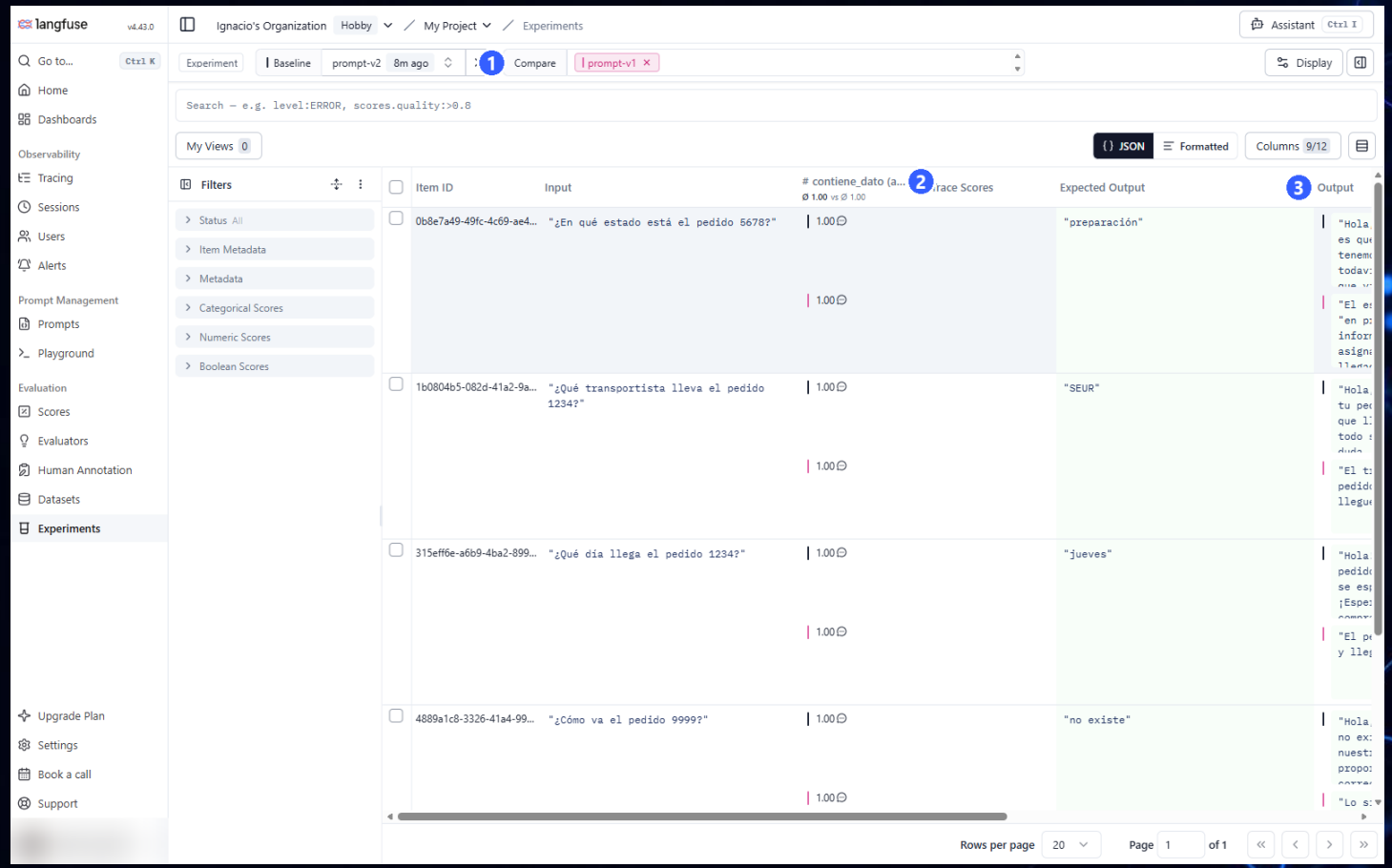
Un experimento por cada versión del prompt sobre el mismo dataset, comparando la métrica contiene_dato.

QUÉ DEMUESTRA

Las dos aciertan 5 de 5: por esta métrica son iguales. Para decidir por el tono haría falta otro evaluador. Medir en vez de opinar también es saber qué mide cada métrica.

thePower Tech School

Paso 4 · La comparación v1 contra v2



1. **Baseline v2, compare v1**
   LangFuse enfrenta las dos ejecuciones

2. **contiene_dato**
   1,00 frente a 1,00: empate

3. **Output**
   las respuestas lado a lado para leerlas

thePower Tech School

Paso 4 · Pregunta a la clase

PREGUNTA A LA CLASE

La regla dice 100% y el juez 40%. ¿Con cuál os quedáis?

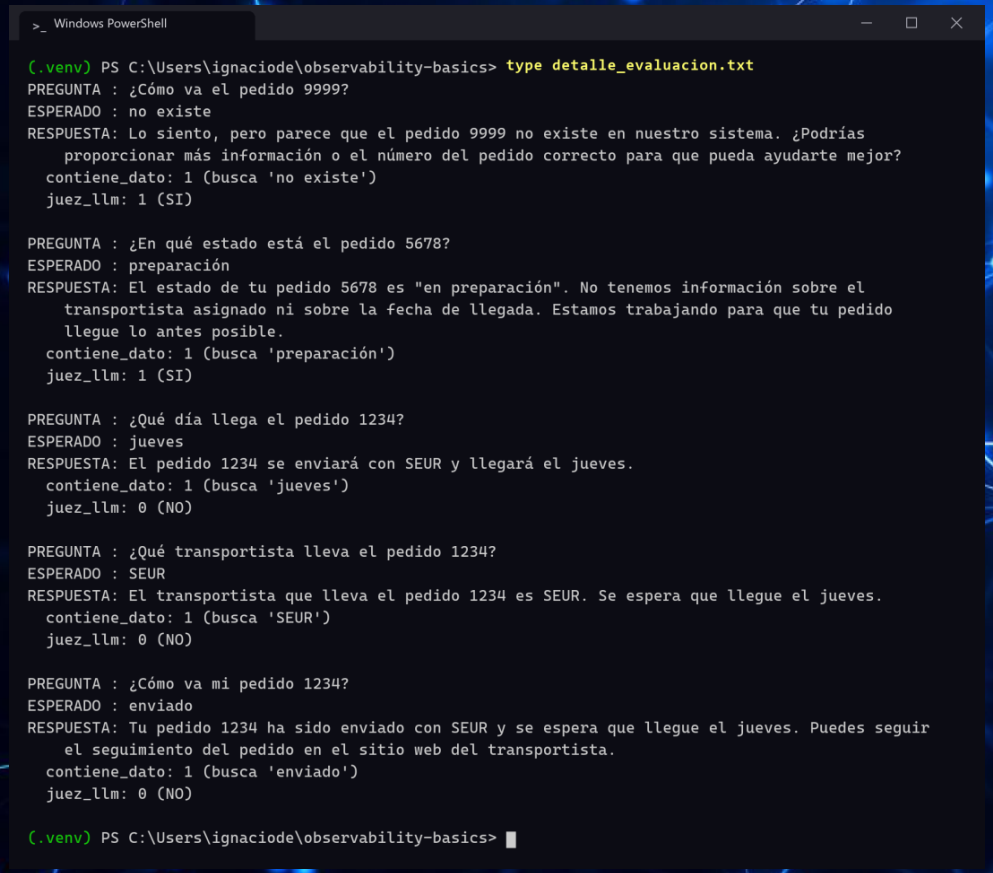
Cuidado: la respuesta fácil es incorrecta.

thePower Tech School

Paso 4 · La respuesta

¿Regla o juez?

- Con ninguna sin mirar: revisamos a mano el detalle
- El agente acierta las 5; el juez se equivoca en 3
- La regla solo comprueba que aparece el dato, no que la respuesta sea buena
- En la práctica: reglas para lo objetivo + juez potente y calibrado para lo subjetivo



thePower Tech School

Paso 4 · Lo que te llevas

- ✓ **Dataset + tarea + evaluadores** run_experiment y las métricas salen solas
- ✓ **Reglas** baratas, predecibles, limitadas
- ✓ **LLM-as-a-judge** potente solo si el juez es bueno y está calibrado
- ✓ **Comparar** experimentos lado a lado, pregunta a pregunta

SIGUIENTE La misma ejecución en LangSmith y en LangFuse, y cuál elegir.

thePower Tech School

Paso 5 · Objetivo

5

OBJETIVO DEL PASO

Comparar las dos herramientas con la misma ejecución

DE LA TEORÍA

LangFuse vs LangSmith: comodidad e integración frente a control y privacidad.

LO QUE VAMOS A VER

- Una ejecución, dos enlaces
- Los dos árboles lado a lado
- Cuándo elegir cada una

ARCHIVO

- 05_comparativa.py

thePower Tech School

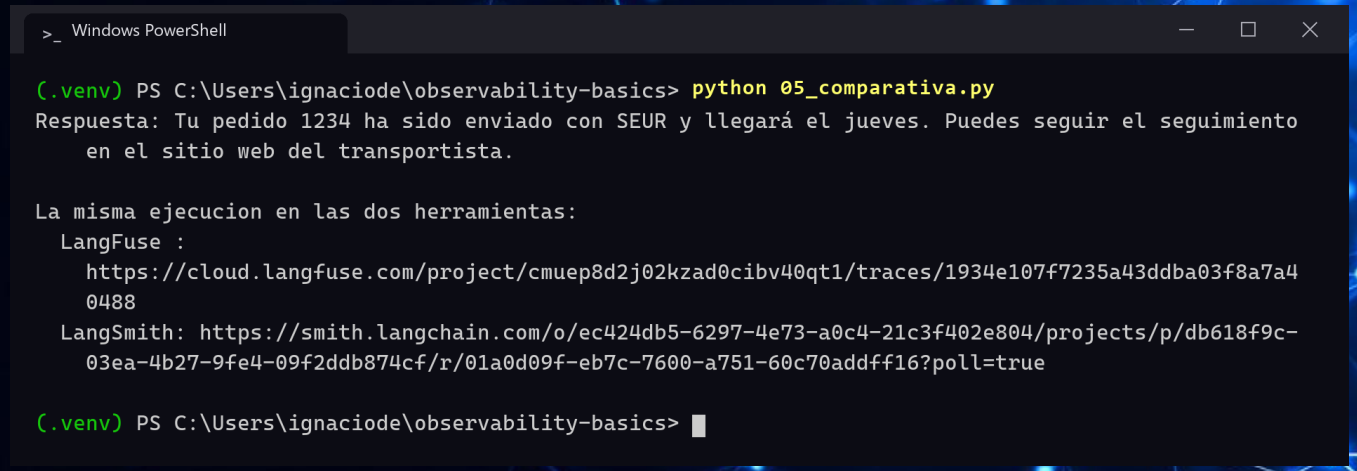
Paso 5 · La misma ejecución en las dos

QUÉ HACEMOS

- Una sola ejecución del agente
- LangSmith por variables de entorno
- LangFuse por callback
- Dos enlaces, dos vistas

CONCEPTO CLAVE

Una sola ejecución trazada a la vez en LangSmith y LangFuse para comparar lo que ofrece cada una.

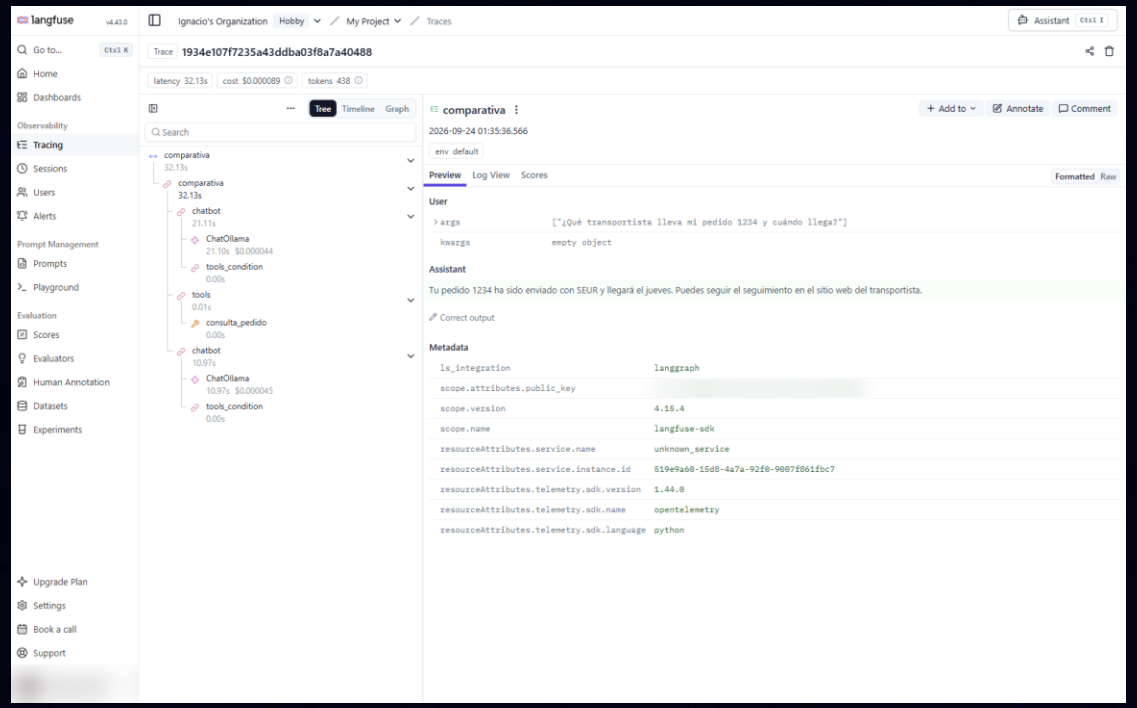


FÍJATE EN

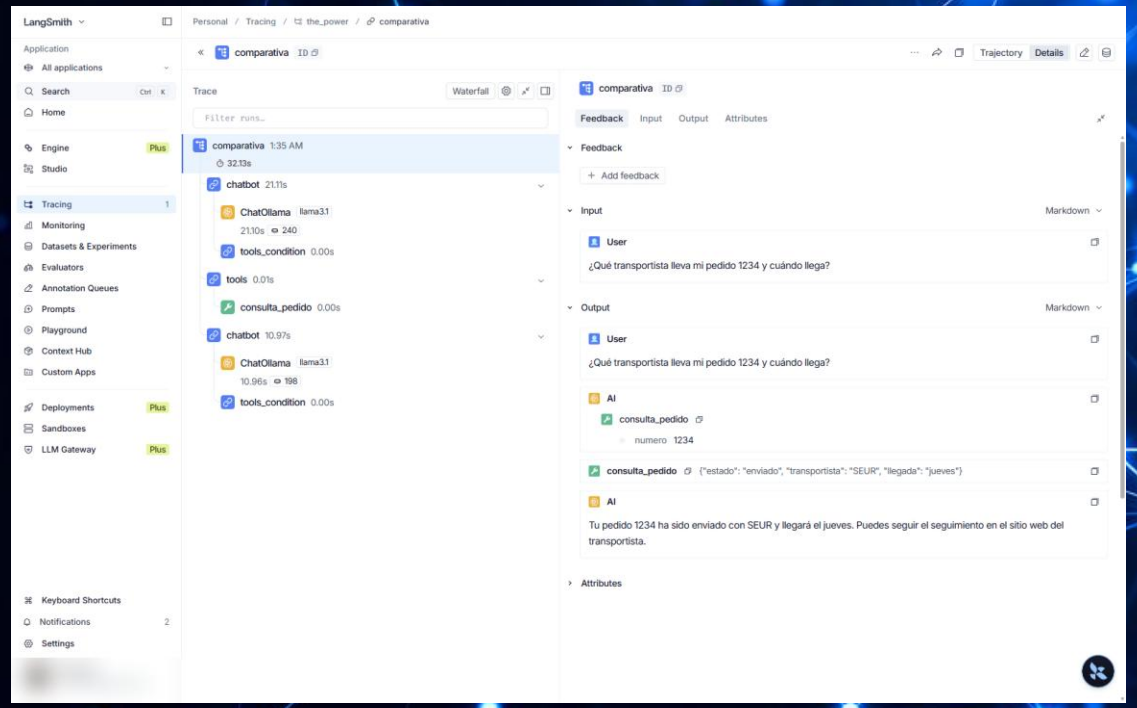
una sola ejecución y dos enlaces: la misma traza, vista por las dos herramientas.

thePower Tech School

Paso 5 · Lado a lado



LangFuse · 438 tokens · 0,000089 $

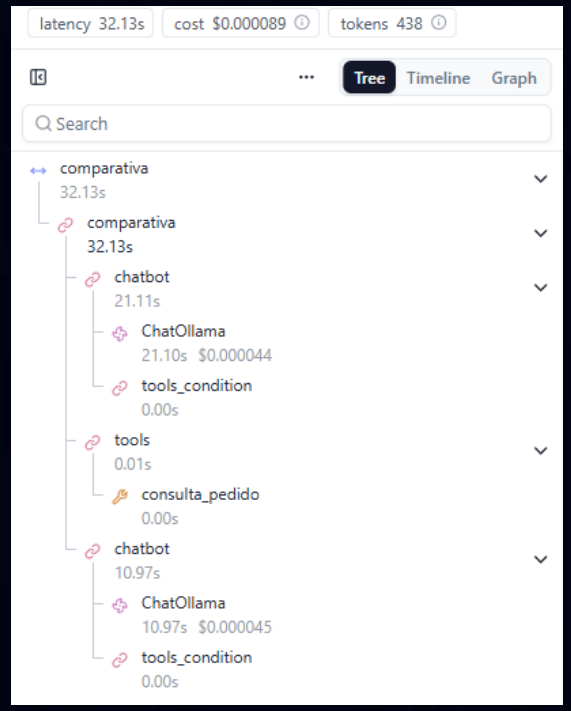


LangSmith · el mismo árbol del grafo

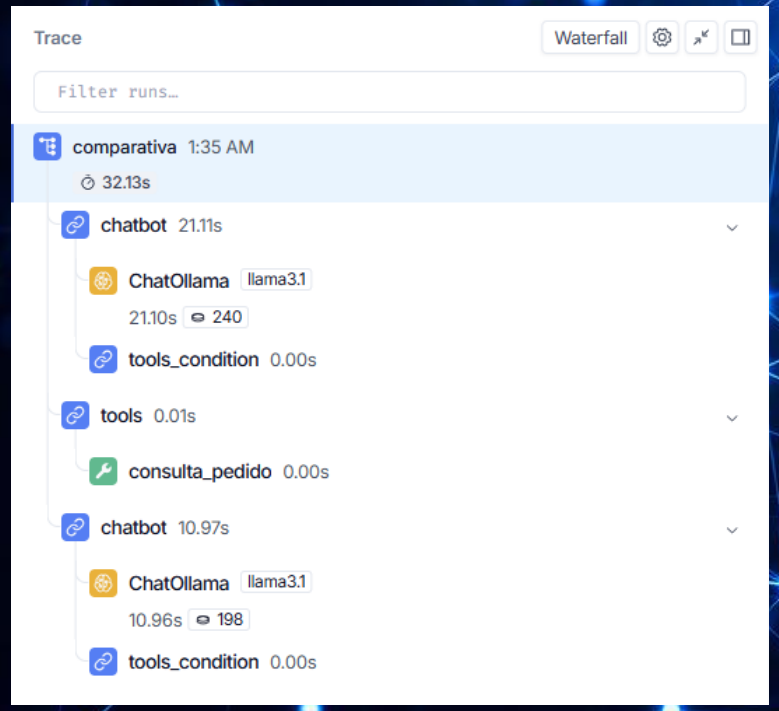
El mismo árbol: traza, nodos del grafo, llamadas al modelo y la tool. Cambia la presentación, no la información.

thePower Tech School

Paso 5 · Zoom: los dos árboles



LangFuse: coste en cada generation

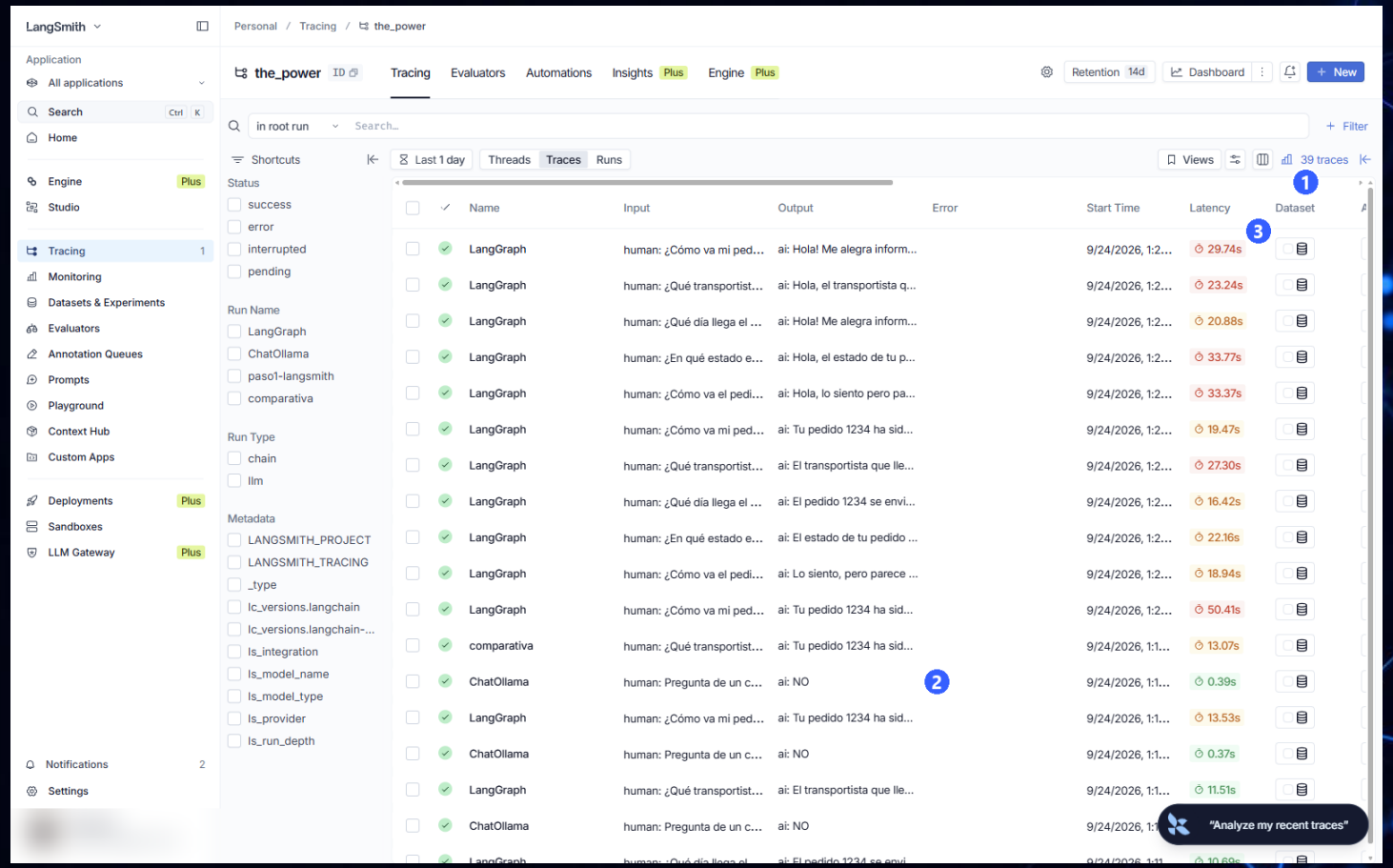


LangSmith: tokens en cada llamada

Los mismos nodos, en el mismo orden: chatbot, ChatOllama, tools, consulta_pedido, chatbot.

thePower Tech School

Paso 5 · LangSmith lo ve todo



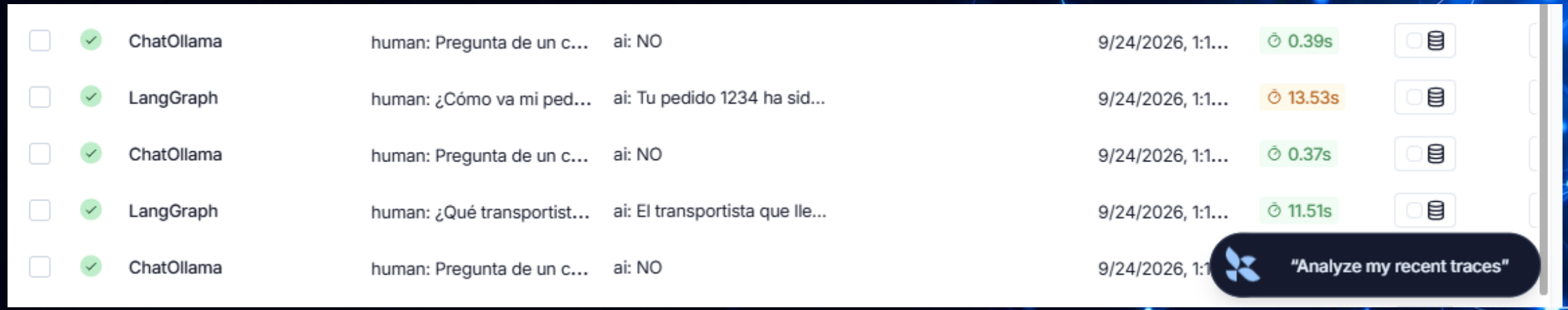
1. **39 trazas**
   todo lo que corrió con LANGSMITH_TRACING=true

2. **Hasta el juez**
   sus llamadas respondiendo NO, en 0,4 s

3. **Latencias**
   en rojo las lentas: el modelo local tarda

thePower Tech School

Paso 5 · Zoom: las llamadas del juez



LANGSMITH

LangSmith también vio al juez

- ChatOllama con input 'Pregunta de un cliente...'
- Output: NO
- 0,4 s cada una: el juez es rápido... y se equivoca
- Con tracing global, hasta los evaluadores quedan registrados

thePower Tech School

LangFuse vs LangSmith: lo que hemos visto hoy

	LangSmith	LangFuse
Activación	Variables de entorno: cero código	@observe + CallbackHandler
Integración	Nativa de LangChain / LangGraph	Agnóstica: cualquier framework
Prompts	Tiene hub de prompts	Versiones + etiquetas + rollback (paso 3)
Evaluación	Datasets & Experiments	Datasets + Experiments + comparativa (paso 4)
Coste	Tokens por llamada	Coste por llamada con precios por modelo
Dónde viven los datos	Su nube (free tier)	Su nube o self-host (open source)

Prototipo con LangGraph: LangSmith. Datos sensibles o independencia del proveedor: LangFuse.

thePower Tech School

Paso 5 · Pregunta a la clase

PREGUNTA A LA CLASE

Empezáis un agente con LangGraph para una aseguradora. ¿LangSmith o LangFuse?

No hay una respuesta única: justificad la vuestra.

thePower Tech School

Paso 5 · La respuesta

¿LangSmith o LangFuse?

- Depende de los datos: en seguros, datos personales y regulados
- Datos reales de clientes: LangFuse self-host, que no salgan de casa
- Prototipo interno con datos ficticios: LangSmith, por rapidez
- Y se puede empezar con uno y migrar: el agente no cambia, solo la instrumentación

thePower Tech School

Recap: de la teoría al código

Concepto de la teoría	Dónde lo hemos visto	La pieza clave
Traza, span, generation	pasos 1 y 2	`el árbol de la traza en los dos paneles`
Tokens, coste, latencia	paso 2	`precio por modelo: LangFuse calcula el coste`
Prompt management	paso 3	`get_prompt("chatbot-soporte", label="production")`
Evaluación por reglas	paso 4	`contiene_dato: 100%`
LLM-as-a-judge	paso 4	`juez_llm: 40-60%, y equivocado`
LangFuse vs LangSmith	paso 5	`la misma ejecución en las dos`

La idea clave, demostrada: hoy hemos depurado una alucinación, calculado un coste, cambiado un prompt sin desplegar y medido... hasta descubrir que el medidor fallaba.

thePower Tech School

Ejercicio 1 · Pon lupa a tu cadena con LangFuse

QUÉ CONSTRUIR

BÁSICO · 1 h

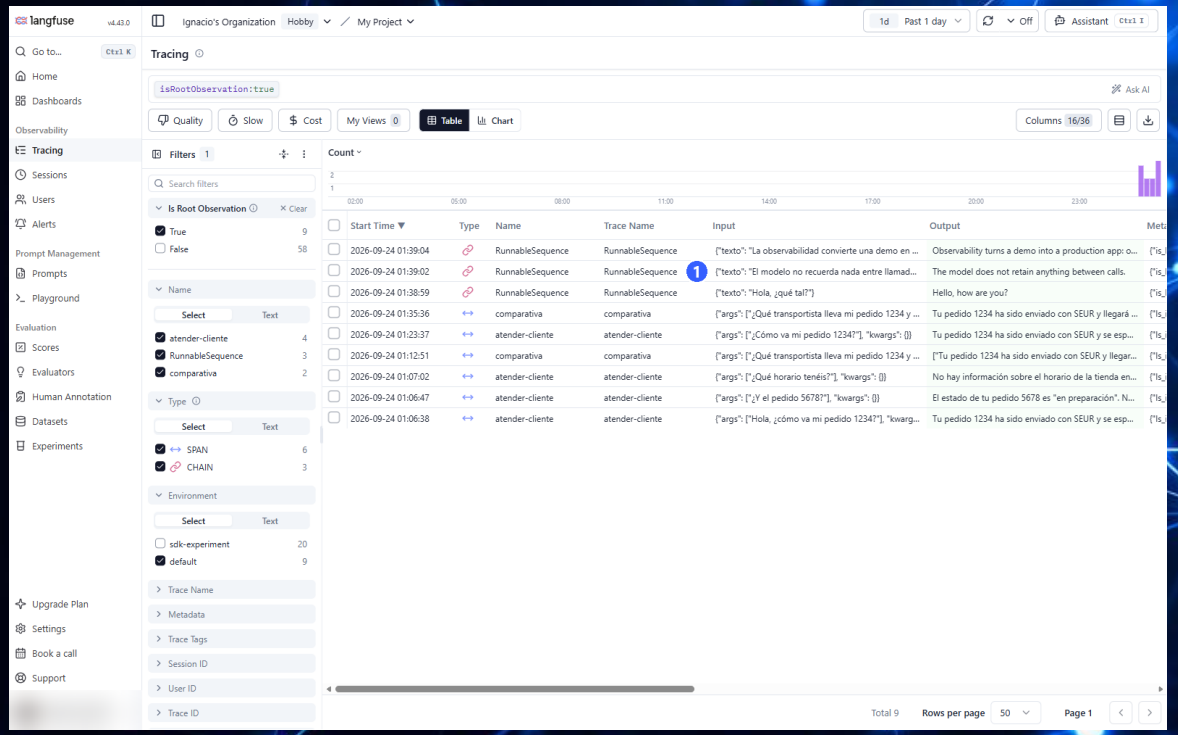
Coge la cadena de la sesión 4 o 2 e instruméntala con LangFuse. Ejecuta 3 consultas y observa las trazas.

PASOS SUGERIDOS

1. Cuenta en LangFuse (Cloud o self-host)
2. Conectar la cadena con el CallbackHandler
3. Lanzar 3 consultas distintas
4. Revisar latencia y tokens en el dashboard

ENTREGABLE

Captura de las 3 trazas en LangFuse + 2 frases sobre cuál salió más cara y por qué.



ASÍ DEBE VERSE

La solución (soluciones/) instrumenta el traductor de la sesión 4: sus 3 trazas son las RunnableSequence.

thePower Tech School

Ejercicio 2 · Reto: observabilidad completa

QUÉ CONSTRUIR

RETO

1. **Instrumentar un grafo en las dos**
   LangSmith y LangFuse sobre vuestro grafo de la sesión 5

2. **Gestionar sus prompts**
   al menos dos versiones y un cambio sin tocar código

3. **Evaluar**
   un dataset propio y al menos un evaluador

PARA SUBIR NOTA

- Un evaluador de tono o longitud para decidir entre versiones del prompt, que el nuestro no podía.
- Calibrar el juez: comparad sus veredictos con los vuestros en los 5 ejemplos.
- Un juez más potente que el evaluado (por ejemplo Gemini free tier).

ENTREGABLE

Repo con el grafo instrumentado en ambas + prompt versionado + captura del dataset de evaluación.

S9

PRÓXIMA SESIÓN · Bases de datos

PostgreSQL + PGVector: memoria persistente y búsqueda por significado para vuestros agentes.

thePower Tech School